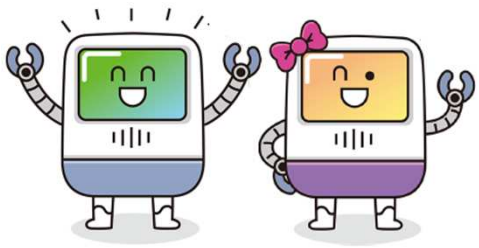
딥러닝

5장 신경세포와 퍼셉트론



05

신경세포와 퍼셉트론

학습목차

01 뇌와 신경세포 그리고 퍼셉트론
02 퍼셉트론의 표현 방법
03 XOR 문제와 다층 퍼셉트론
04 퍼셉트론 학습시키기

학습목표

- 인간과 동물의 신경세포가 가지는 특징을 이해하고 이를 흉내 낸 퍼셉트론의 구조를 살펴본다.
- 퍼셉트론을 이루는 입력값, 가중치, 신호 합산, 활성화 함수의 기능을 이해한다.
- 행렬을 이용하여 퍼셉트론의 구조를 간략화하는 방법을 알아본다.
- OR, AND 논리회로를 단층 퍼셉트론으로 구현하는 방법과 XOR 논리 회로의 특징을 이해한다.
- 다층 퍼셉트론을 이해하고 퍼셉트론을 학습시키는 방법을 이해한다.



뇌와 신경세포 그리고 퍼셉트론
- 경이로운 인간의 뇌와 지능의 신비
- 신경세포를 흉내 낸 프로그램인 퍼셉트론
- 퍼셉트론과 연결주의의 등장

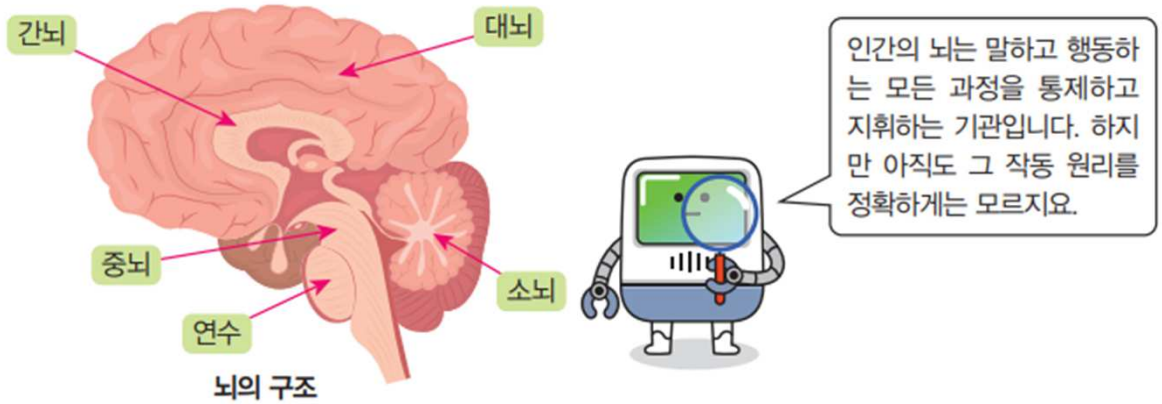
퍼셉트론의 표현 방법
- 행렬을 이용한 퍼셉트론 표현 방법
- 신호의 전달 여부를 결정하는 임계값
- 퍼셉트론 프로그램 만들기
- OR 논리 연산을 퍼셉트론으로 구현하자
- 편향값을 추가하자
- 편향값을 추가하여 활성화 함수를 간단하게 만들자
- 편향값을 가진 퍼셉트론으로 구현하자

XOR 문제와 다층 퍼셉트론
- 논리 연산의 결과를 시각화하자
- 인공지능 연구의 침체기를 가져온 XOR 문제
- 은닉 노드를 가진 다층 퍼셉트론의 등장
- 다층 퍼셉트론으로 해결할 수 있는 문제들

퍼셉트론 학습시키기
- 퍼셉트론과 학습의 원리
- 퍼셉트론의 학습을 위한 행렬표현 방법
- 동시에 여러 가지 작업을 처리하는 병렬 처리 시스템
- 다층 퍼셉트론의 가중치들은 어떻게 학습시킬까

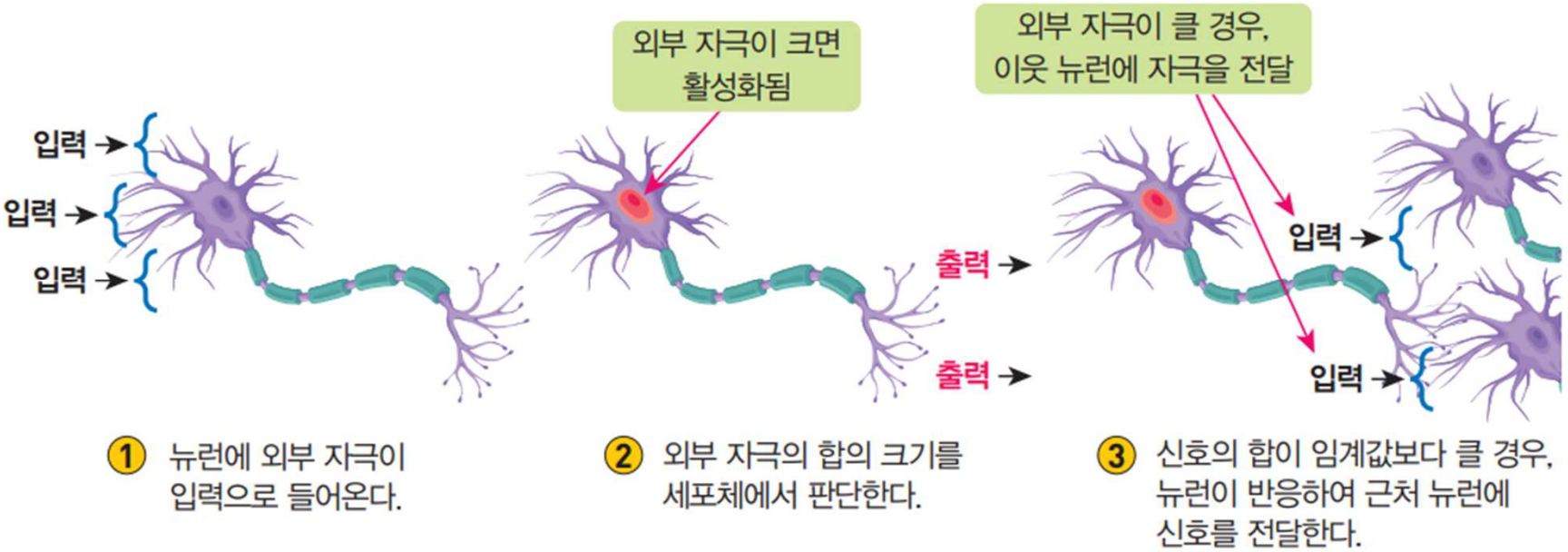
01 뇌와 신경세포 그리고 퍼셉트론

- 경이로운 인간의 뇌와 지능의 신비
  - 인간과 동물의 사고를 담당하는 신체 기관은 **뇌** brain 이다. 인간의 뇌(腦)는 약 1.5kg 정도에 불과한 근육도 뼈도 없는 거대한 단백질 덩어리이지만, **감각을 느끼며, 말을 하고, 생각하고, 행동하는 모든 과정을 통제하고 지휘하며 조정하는 곳**이다.
  - 뇌는 크게 대뇌, 중뇌, 소뇌, 간뇌, 연수 등으로 구분되며 총괄적인 지휘를 하는 부분은 대뇌로, 기억과 감각을 관리하는 곳이다. 인간의 뇌가 동물의 뇌와 크게 차이가 나는 부분은 대뇌 피질인데, 이 부분은 다른 동물과 달리 매우 발달되어 있으며 많은 주름을 가지고 있다.
  - 뇌의 대부분은 신경세포 덩어리이며 신경세포가 밀집되어 연결된 구조를 가지고 있다. 하나의 신경세포는 세포핵, 수상돌기, 축삭, 축삭 말단과 같은 구조를 가진다.



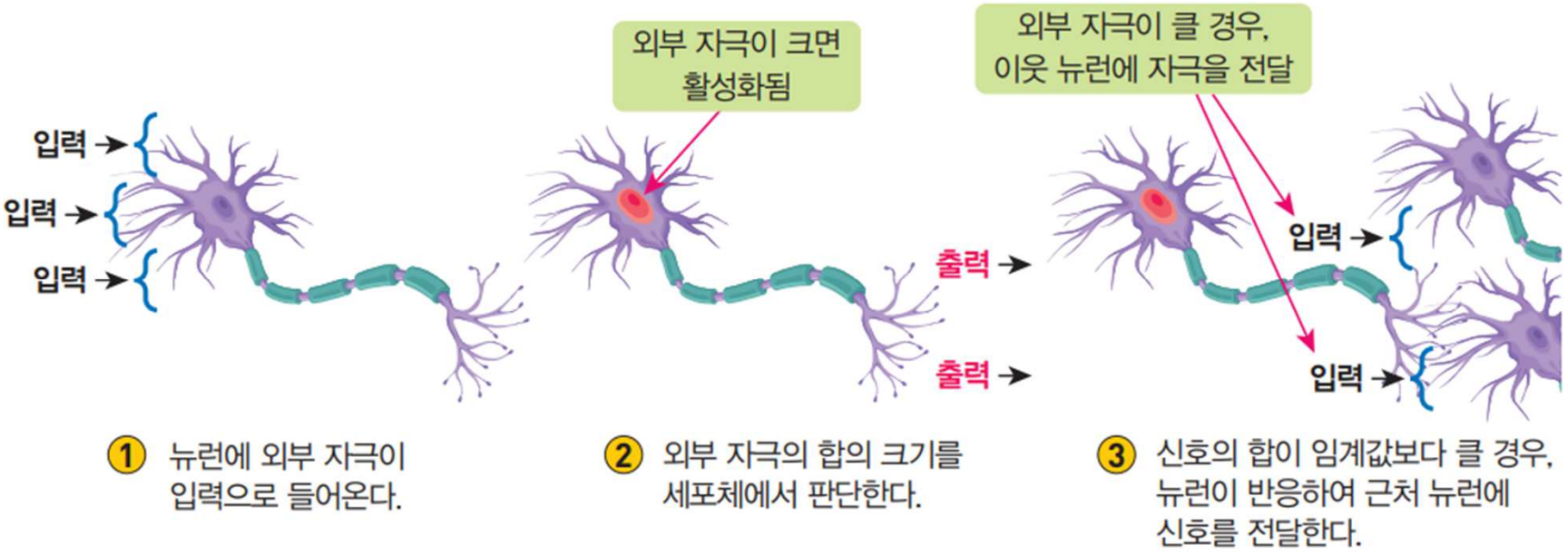
경이로운 인간의 뇌와 지능의 신비

- 생물학자들의 연구에 의하여 뇌가 가지는 구조의 개략적인 특징이 알려져 있다.
  - **뉴런** neuron 이라는 신경세포가 다른 신경 세포와 연결되어 있고, 나트륨, 칼륨 등의 물질을 이용하여 화학적 신호를 주고받는다
  - 신경전달 물질에 의하여 그림과 같이 한 뉴런은 외부 자극을 인접한 뉴런에 전달하게 된다



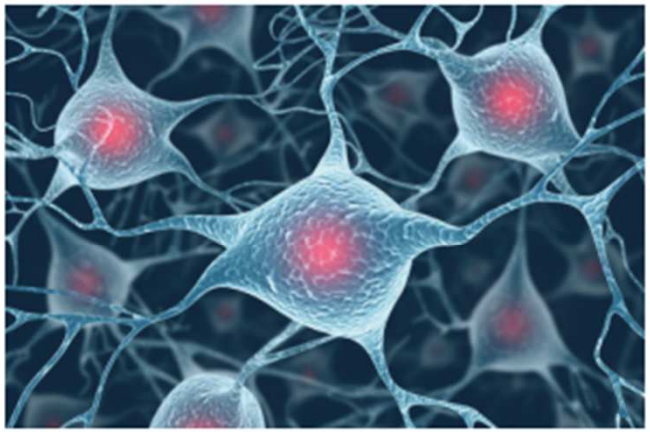
경이로운 인간의 뇌와 지능의 신비

- 방대한 연결 구조를 가지는 신경세포에서 수상돌기는 그림 1 과 같이 다른 신경세포로부터 신호를 전달받아서(외부의 자극이 입력 신호에 해당한다), 이 외부 자극의 합의 크기를 세포체에서 판단한다(그림 2). 다음으로 이 신호의 합이 임계값보다 클 경우, 뉴런이 반응하여 축삭 말단을 통해서 다른 신경세포로 신호를 전달한다(그림 3).

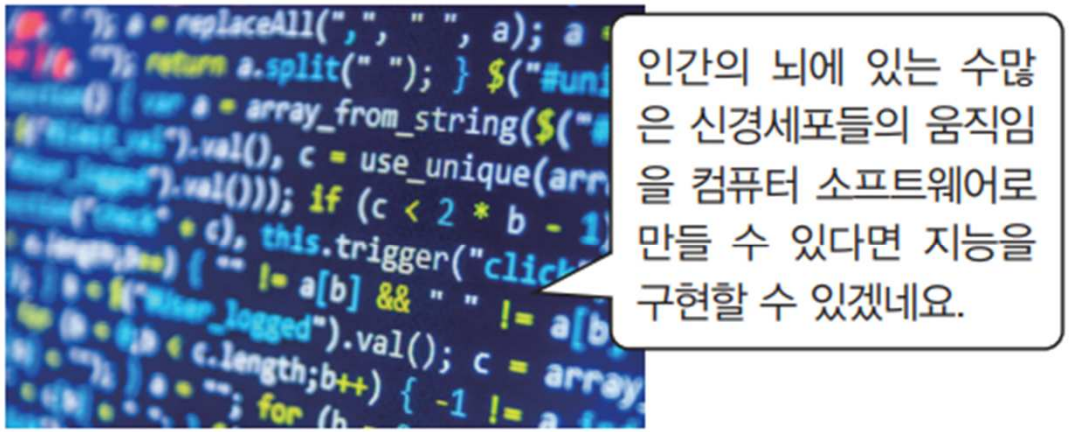


경이로운 인간의 뇌와 지능의 신비

- 이러한 인간의 신경세포가 가지는 구조를 컴퓨터로 나타내면 지능을 구현할 수 있을 것이다
  - 연결주의라고 함
  - 뉴런(신경세포)을 퍼셉트론(인공적인 신경세포)으로 구현하려는 시도



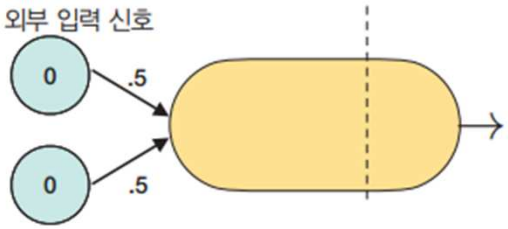
인간의 신경세포



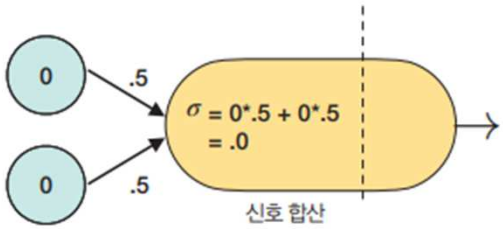
컴퓨터 소프트웨어

경이로운 인간의 뇌와 지능의 신비

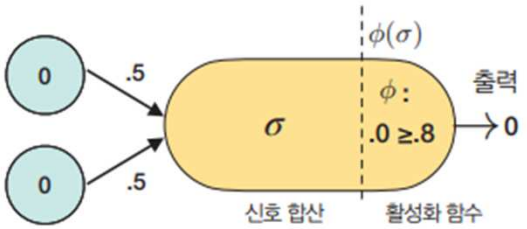
- 이전의 그림과 같이 외부 신호가 퍼셉트론으로 들어오면(그림 1), 이 외부 신호와 가중치를 곱한 합의 크기인 신호 합산 값을 구한다(그림 2).
  - 다음으로 이 신호 합산 값이 임계값보다 클 경우, 퍼셉트론이 이 값을 인접 퍼셉트론으로 전달한다(그림 3).
  - 이와 같은 퍼셉트론의 연결을 통해서 인공적으로 뉴런을 흉내 낸다
  - 컴퓨터를 사용하여 흉내내어야 하므로 모든 자극값(외부 신호)과 가중치, 출력(신호 합산)은 수치값으로 표현된다



① 퍼셉트론에 외부 신호가 입력으로 들어온다.



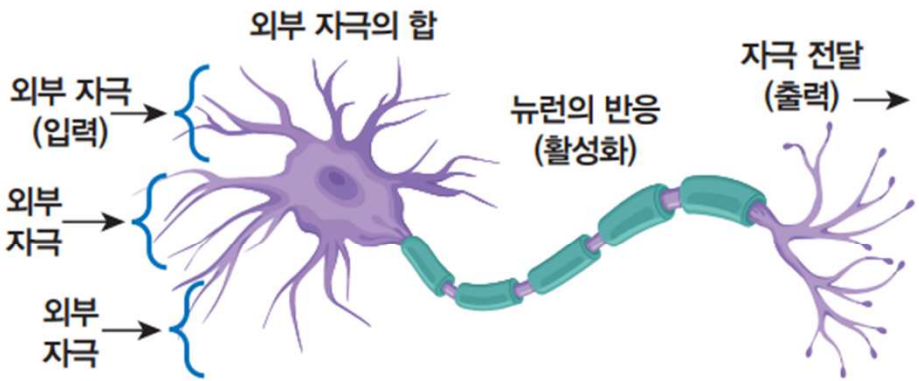
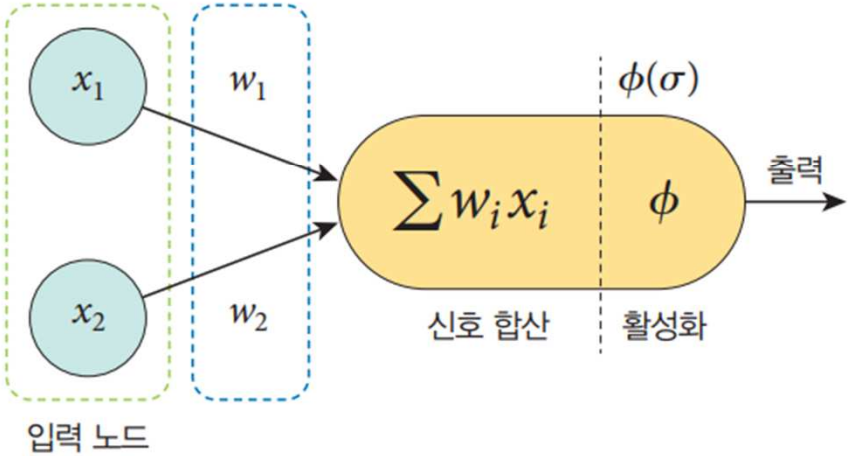
② 외부 입력 신호와 가중치를 곱한 합의 크기인 신호 합산 값을 구한다.



③ 신호의 합이 임계값보다 클 경우, 퍼셉트론이 신호를 출력한다. 이 출력은 인접 퍼셉트론으로 전달된다.

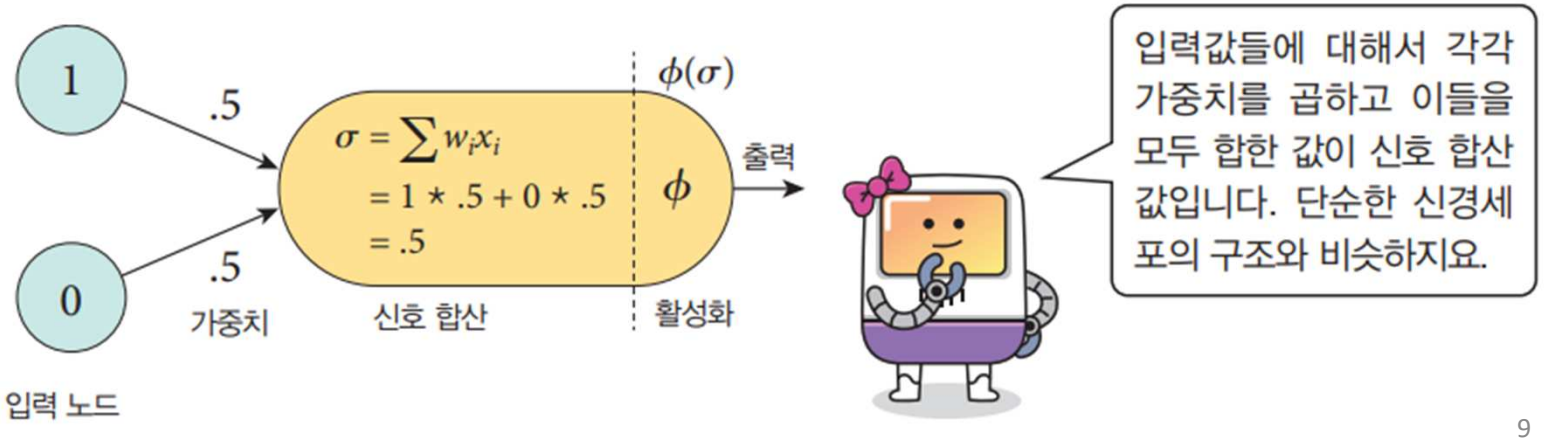
신경세포를 흉내 낸 프로그램인 퍼셉트론

- 신경세포(뉴런)과 인공 신경세포(퍼셉트론)의 비교

	단위 구조	의미
신경세포의 구조		신경세포에 들어온 외부 자극에 대해 반응하여 외부 신경세포에 자극을 전달하는 구조이다.
인공 신경세포의 구조		입력 노드로 들어온 값을 가중치 값에 곱한다. 이 값을 합한 후 그 값이 일정한 값 이상이 되면 외부 노드에 출력값으로 내보낸다.

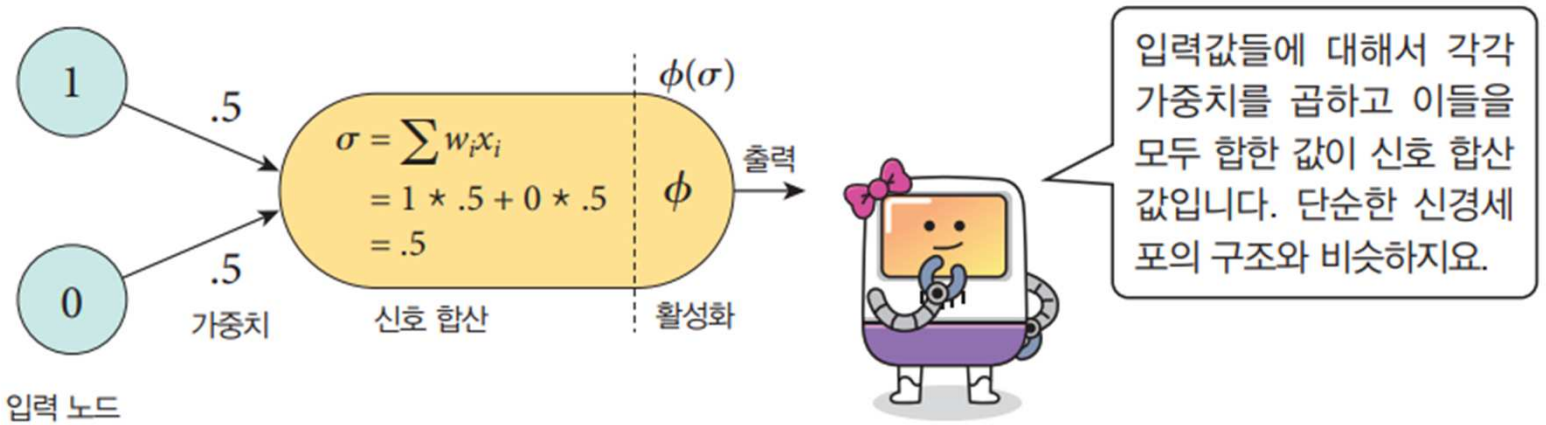
신경세포를 흉내 낸 프로그램인 퍼셉트론

- 외부로부터 입력된 자극을 x_1, x_2와 같은 수치값으로 할당한 후, 이 수치값을 ω_1, ω_2의 가중치에 곱한 다음 이 신호의 합을 구한다.
  - x_1, x_2 값을 가지는 원은 외부의 자극 값으로 **노드** node 라고도 한다. 이 외부 자극에 곱해지는 값은 파라미터 혹은 가중치로 불리는데, 가중치의 영어인 weight의 앞문자 w에 아래 첨자를 이용하여 ω_i와 같이 표기한다.
  - 그림에서 첫 번째 노드의 값은 1이고, 이 노드와 연결된 노드의 가중치는 .5이다. 따라서 1*.5가 $x_1\omega_1$이다.
  - 두 번째 노드의 값은 0이고, 이 노드와 연결된 가중치는 .5이다. 따라서 0*.5가 $x_2\omega_2$이다. 첫 번째 노드와 가중치, 두 번째 노드와 가중치의 곱의 합은 $\omega_1x_1 + \omega_2x_2$로 나타낼 수 있다



신경세포를 흉내 낸 프로그램인 퍼셉트론

- 신호 합산 값 $\sum \omega_i x_i$는 간단하게 σ로도 표기할 수 있다.
  - 신호 합산 값 σ는 특정한 활성화 함수 $\phi()$의 입력으로 들어가서 이 값이 임계치를 넘을 경우 출력값을 내보낸다.
  - 이 출력값은 다시 인접 퍼셉트론의 입력으로 전달되는 구조를 가지고 있다.
  - 이 구조는 입력 수치값과 가중치의 곱, 이들의 합 그리고 활성화 함수의 호출로 이루어지므로 컴퓨터 프로그래밍을 통해서 쉽게 구현할 수 있다.



신경세포를 흉내 낸 프로그램인 퍼셉트론

- 입력값, 출력값으로 신호 합산 값을 구해보자

도전 문제 5.1: 입력 노드의 값과 가중치를 가진 퍼셉트론

상 중 하



1. 입력 노드의 값이 (1, 1)이고 가중치가 (.5, .5)인 퍼셉트론이 있다. 이 값들의 신호 합산 값은 얼마인가?

2. 입력 노드의 값이 (2.2, 1.3)이고 가중치가 (.5, −.5)인 퍼셉트론이 있다. 이 값들의 신호 합산 값은 얼마인가?

3. 입력 노드의 값이 (1.2, 2.1, 1.3)이고 가중치가 (1.0, .5, −.2)인 퍼셉트론이 있다. 이 값들의 신호 합산 값은 얼마인가?

퍼셉트론과 연결주의의 등장

- 연결주의의 등장
  - 1958년 미국 해군 연구실
  - 프랭크 로젠블랫에 의해 최초로 공개됨
  - 당시 인공지능의 새로운 장을 연 기술로 각광을 받았음
  - 인간과 유사하게 **신경세포의 구조를 인위적으로 만들어서 지능을 구현하려는 연구 방향을 연결주의**라고 함

새로운 해군 장치는 경험을 통해 배운다

읽을 수 있고 똑똑해지도록 설계된 컴퓨터의 배아를 심리학자가 보여 주었다.

워싱턴, 7월 7일 (UPI) - 해군이 오늘 **걷고, 말하고, 보고, 쓰고, 번식하고, 스스로의 존재를 의식할 수 있는 전자 컴퓨터의 배아를 공개**했다. 해군이 기자들에게 보인 시연에서 기상 부서의 2백만 달러짜리 컴퓨터 '704'는 50회의 시도 끝에 오른쪽과 왼쪽을 구분하는 법을 익혔다. 해당 부서는 이 방식을 적용하여 읽고 쓰며 생각하는 퍼셉트론 기계를 만들 것이라고 밝혔다. 1년 정도의 기간으로 10만 달러의 비용을 들여 완성할 것으로 예상된다. 퍼셉트론의 설계자인 **프랭크 로젠블랫** 박사가 시연을 했다. 그는 이 기계가 처음으로 인간의 뇌와 같이 사고할 수 있는 장치가 될 것이라고 말했다. 그리고 인간처럼 퍼셉트론도 처음에는 실수를 하겠지만, 경험이 늘수록 더 똑똑해질 것이라고도 했다. 로젠블랫 박사는 버팔로에 있는 코넬 항공 연구소의 심리학 연구자이며, 퍼셉트론을 다른 행성에 발사하여 기계로 된 우주 탐사자로 쓸 수 있다고 했다.



NEW NAVY DEVICE LEARNS BY DOING

Psychologist Shows Embryo of Computer Designed to Read and Grow Wiser

WASHINGTON, July 7 (UPI) —The Navy revealed the embryo of an electronic computer today that it expects will be able to walk, talk, see, write, reproduce itself and be conscious of its existence.

The embryo—the Weather Bureau's $2,000,000 "704" computer—learned to differentiate between right and left after fifty attempts in the Navy's demonstration for newsmen.

The service said it would use this principle to build the first of its Perceptron thinking machines that will be able to read and write. It is expected to be finished in about a year at a cost of $100,000.

Dr. Frank Rosenblatt, designer of the Perceptron, conducted the demonstration. He said the machine would be the first device to think as the human brain. As do human beings, Perceptron will make mistakes at first, but will grow wiser as it gains experience, he said.

Dr. Rosenblatt, a research psychologist at the Cornell Aeronautical Laboratory, Buffalo, said Perceptrons might be fired to the planets as mechanical space explorers.

해군은 퍼셉트론이 인간의 훈련과 통제 없이도 주변 환경을 파악하고 인식하며 분별할 수 있는 **최초의 무생물 메커니즘**이 될 것이라고 말했다.

... <중간 생략> ...

퍼셉트론과 연결주의의 등장

- 1950년대의 연구 주류는 기호주의임
  - 기호주의자들은 로젠블랫이 과학적 기준을 따르지 않고, 언론을 활용해 '과장된 주장'을 한다고 보았다. 그리고 이에 대한 비판의 최전선에 마빈 **민스키 교수**가 나섬

NOTE: 기호주의 vs 연결주의



인공지능 연구에는 두 가지의 큰 흐름이 있는데 하나는 기호주의이고 다른 하나는 연결주의이다. 이들은 다음과 같은 특징을 가지고 있다.

- **기호주의:** 컴퓨터가 작동하는 방식인 기호와 규칙을 사용하여, 인간 사고와 최대한 유사한 규칙을 만들어서 인간의 지능을 흉내 내고자 하는 연구방식이다. 전문가 시스템, 수학적 정리 증명, 게임, 자연어 처리 등의 분야에서 응용된다.
- **연결주의:** 입력값과 가중치 그리고 이들의 가중합을 출력하여 인접한 신경망의 입력으로 하는 방식인데, 인공지능의 구현을 최대한 인간의 신경세포의 연결 구조와 유사하게 구현하는 방식을 말한다. 최근 인공지능의 주류를 이루며 좋은 성과를 내는 딥러닝은 이 연결주의 방식의 인공지능이다.

02 퍼셉트론의 표현 방법

- 행렬을 이용한 퍼셉트론 표현 방법
  - 간단한 행렬 표현법과 넘파이를 이용해서 신경망의 구조를 간단하게 나타낼 수 있음을 알아보자.
  - 입력 벡터 X와 가중치 W가 있을 경우, 두 벡터의 곱과 이 벡터곱의 합은 다음과 같이 넘파이 연산으로 구할 수 있다.

```
import numpy as np


X = np.array([1, 0])      # 입력 벡터 X
W = np.array([0.5, 0.5])  # 가중치 벡터 W
X * W                     # X와 W의 각 성분끼리 곱함: [1*.5, 0*.5]
```

```
array([0.5, 0. ])
```

```
np.sum(X * W)  # 넘파이 sum 함수를 사용하여 성분의 합 구하기
```

```
0.5
```

```
np.dot(X, W)   # 넘파이 dot 함수를 사용하여 합 구하기(위와 동일한 결과)
```

```
0.5
```

행렬을 이용한 퍼셉트론 표현 방법

- 넘파이를 이용하여 성분합을 구하자

도전 문제 5.2: 퍼셉트론의 성분합 구하기

상 중 하



1. 입력 노드의 값이 (1, 1)이고 가중치가 (.5, .5)인 퍼셉트론이 있다. 이 값들의 신호 합산 값은 얼마인가? 넘파이의 `sum()`과 `dot()` 함수를 이용하여 이 값을 각각 계산하여라.

2. 입력 노드의 값이 (1, 1)이고 가중치가 (.3, .8)인 퍼셉트론이 있다. 이 값들의 신호 합산 값은 얼마인가? 넘파이의 `sum()`과 `dot()` 함수를 이용하여 이 값을 각각 계산하여라.

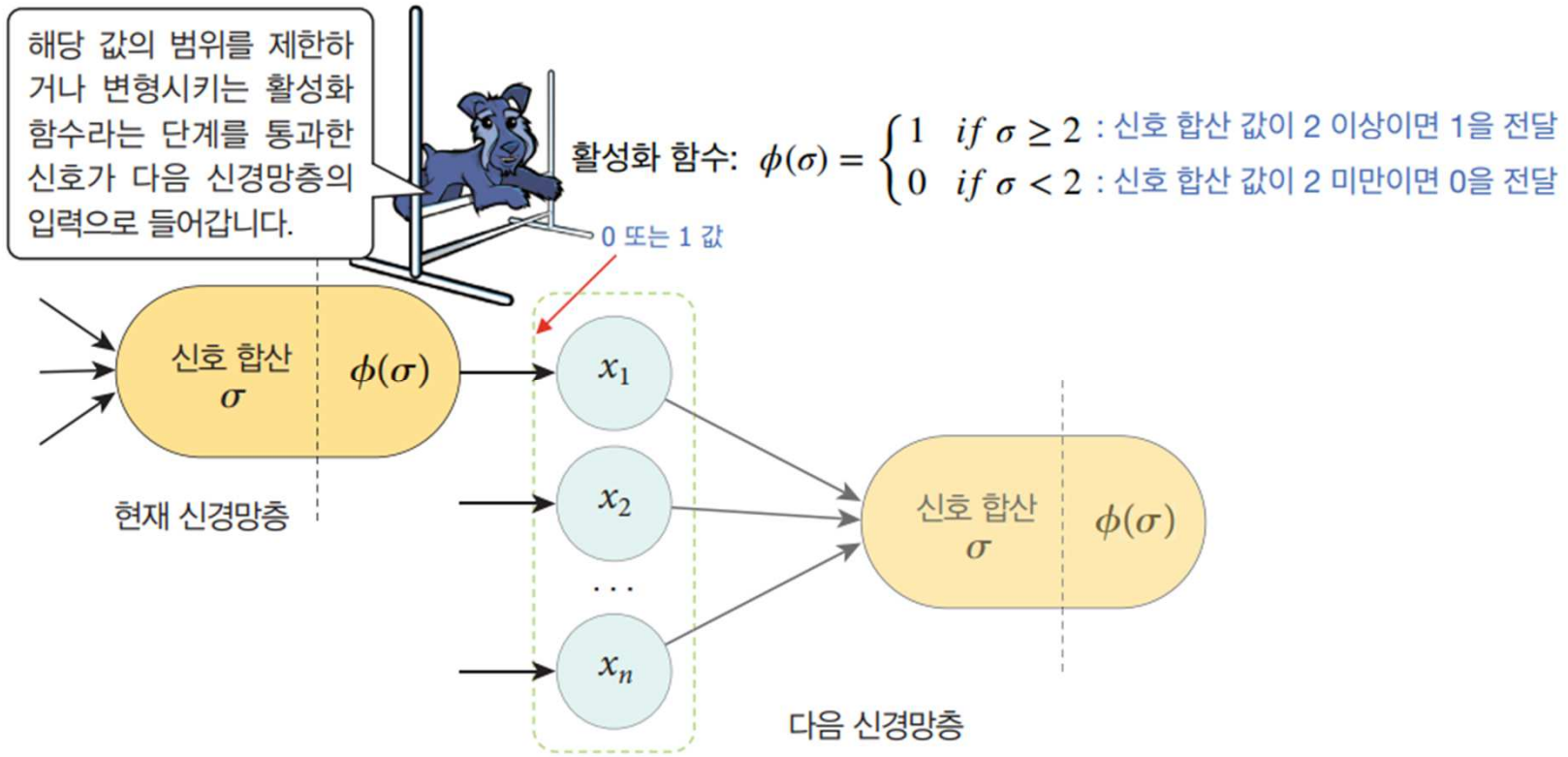
3. 입력 노드의 값이 (1.2, 2.1, 1.3)이고 가중치가 (1.0, .5, −.2)인 퍼셉트론이 있다. 이 값들의 신호 합산 값은 얼마인가? 넘파이의 `sum()`과 `dot()` 함수를 이용하여 이 값을 각각 계산하여라.

신호의 전달 여부를 결정하는 임계값

- **활성화 함수** activation function 는 인공신경망에서 각 퍼셉트론의 출력 값을 결정하는 역할을 함.
  - 이 함수는 입력 신호의 총합이 활성화를 일으키는지에 대한 여부, 즉 한 퍼셉트론이 다음 퍼셉트론에 신호를 전달할 것인지를 결정한다.
  - 활성화 함수의 주요한 목적 중 하나는 비선형성을 신경망에 도입하는 것
  - 선형 함수만 사용한다면, 신경망의 계층 수를 늘리는 것이 의미가 없어지기 때문이다.
  - 활성화 함수가 비선형일 때만 여러 계층을 갖는 신경망이 복잡한 패턴을 학습할 수 있다.
  - 딥러닝을 위한 다양한 활성화 함수들이 존재하며, 각각은 특정 문제나 네트워크 구조에 더 적합할 수 있다.
    - **시그모이드 함수** sigmoid function , ReLU(Rectified Linear Unit), Tanh, Leaky ReLU등이 널리 사용된다. 이들은 각각 서로 다른 수학적 특성과 장단점을 가지고 있다.
  - 이 함수는 앞으로 $\phi()$로 표기할 것이다.

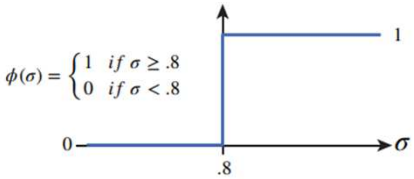
신호의 전달 여부를 결정하는 임계값

- 입력된 신호 합산 값을 다음 퍼셉트론에 전달할지 말지를 결정하기 위해 **임계값** threshold 혹은 문턱값이라고 하는 상수를 사용하는 아주 간단한 계단 함수에 대하여 알아보자



신호의 전달 여부를 결정하는 임계값

- 계단 함수에서 사용되는 이 값은 퍼셉트론에 전달된 신호의 누적 합이 특정한 자극치를 넘어야만 이 신호를 이웃 퍼셉트론으로 전달하는 역할을 담당한다.
  - 다음 그림과 표를 살펴보면 σ 값이 .8보다 작으면 0의 값, .8 이상이면 1의 값을 내보내는 계단 형태의 함수가 있다.
  - 이 계단 함수는 그림의 오른쪽과 같이 점프 높이가 일정 값을 넘으면 통과(1), 그렇지 않으면 탈락(0)이라는 상태를 가지는 높이 뛰기에 비유할 수 있다



σ가 .8보다 크면 1을 내보내는 활성화 함수



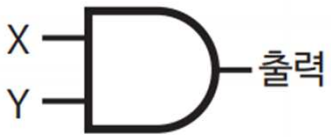
점프 높이가 일정 값을 넘으면 통과(1), 아니면 탈락(0)

• 가중치 ω_1, ω_2는 각각 .5, .5로, 임계값은 .8로 가정함

입력 x_1	입력 x_2	$\sum \omega_i x_i$: 신호의 합 σ	$\phi()$ 함수 : $\sigma \geq .8$	출력
0	0	0×.5 + 0×.5 = .0	거짓(.0 ≥ .8)	0
0	1	0×.5 + 1×.5 = .5	거짓(.5 ≥ .8)	0
1	0	1×.5 + 0×.5 = .5	거짓(.5 ≥ .8)	0
1	1	1×.5 + 1×.5 = 1.0	참(1.0 ≥ .8)	1

신호의 전달 여부를 결정하는 임계값

- 신호의 합과 활성화 함수를 사용하여 AND 연산을 구현하자.

AND		AND 게이트는 2개 이상의 입력에 대하여 1개의 출력을 가짐. 논리곱 연산을 수행하며 입력이 하나라도 0인 경우 0을 출력함.

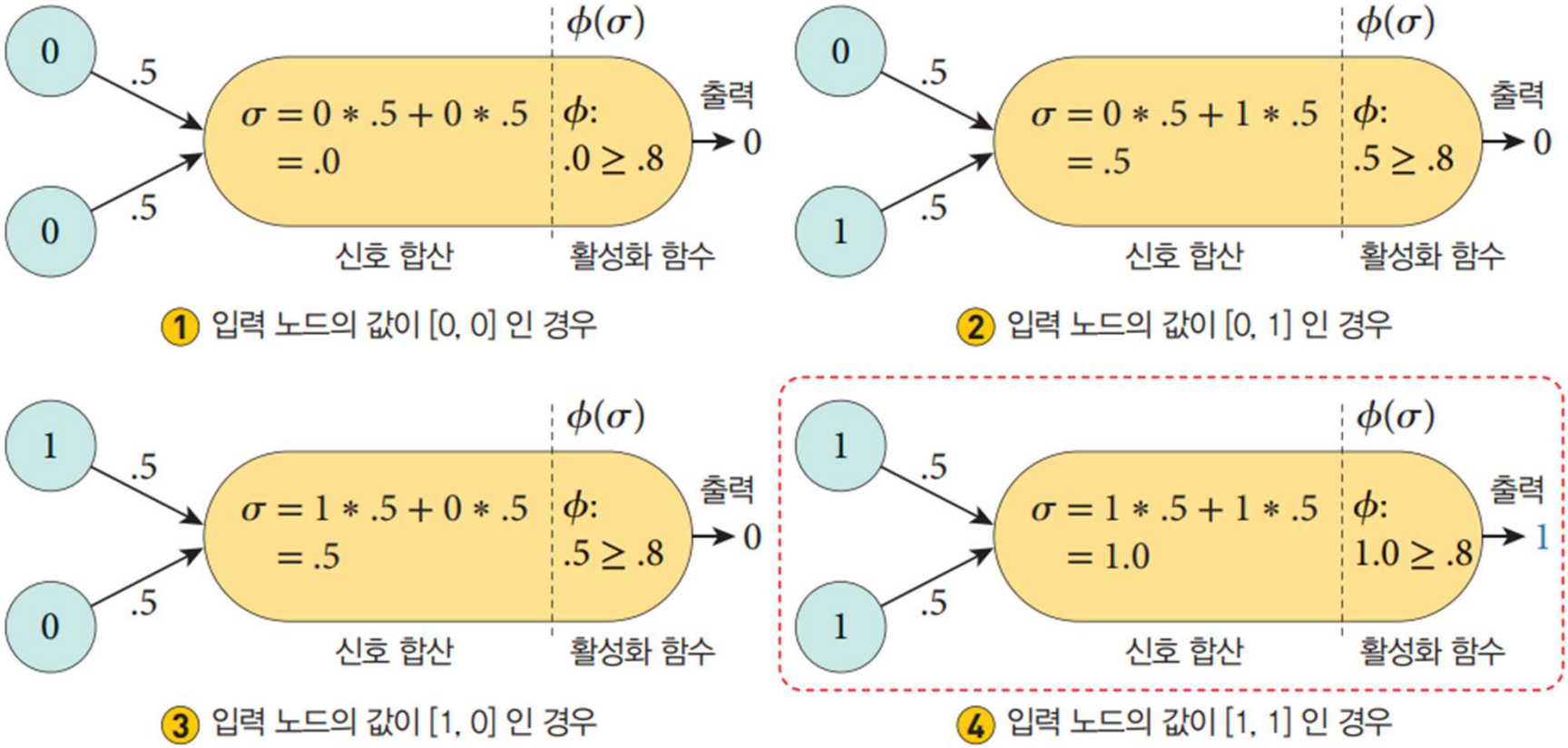
X	Y	출력
0	0	0
0	1	0
1	0	0
1	1	1

• 가중치 ω_1, ω_2는 각각 .5, .5로, 임계값은 .8로 가정함

입력 x_1	입력 x_2	$\sum \omega_i x_i$: 신호의 합 σ	$\phi()$ 함수 : $\sigma \geq .8$	출력
0	0	0×.5 + 0×.5 = .0	거짓(.0 ≥ .8)	0
0	1	0×.5 + 1×.5 = .5	거짓(.5 ≥ .8)	0
1	0	1×.5 + 0×.5 = .5	거짓(.5 ≥ .8)	0
1	1	1×.5 + 1×.5 = 1.0	참(1.0 ≥ .8)	1

신호의 전달 여부를 결정하는 임계값

- 입력과 가중치, 활성화 함수에 따른 출력을 알아보자
  - 입력이 1, 1인 경우에만 출력이 1인 AND 함수가 구현되었다



퍼셉트론 프로그램 만들기

- 넘파이를 사용하여 AND 연산을 수행하는 논리 게이트를 구현하자

```python
import numpy as np

# 가중치와 임계값을 이용한 AND 연산의 구현
def AND_gate(X):
    W = np.array([.5, .5]) # 가중치 값으로 [.5, .5]
    threshold = .8         # 임계값
    sigma = np.sum(X * W)
    if sigma >= threshold: # 반환값 0, 1을 결정
        return 1           # 임계값 이상이면 1(참)을 반환
    else:
        return 0   # 임계값보다 작으면 0(거짓)을 반환

print('AND 논리 연산')
inputs = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
for t in inputs:
    print(t, ':', AND_gate(t))
```

```
AND 논리 연산
[0 0] : 0
[0 1] : 0
[1 0] : 0
[1 1] : 1
```

AND 연산

X	Y	출력
0	0	0
0	1	0
1	0	0
1	1	1

OR 논리 연산을 퍼셉트론으로 구현하자

- 다음은 AND 논리 연산과 OR 논리 연산의 결과를 보여주는 표이다
  - 컴퓨터를 구성하는 기본 연산이다.
  - AND 논리 연산을 수행하는 퍼셉트론에서 임계값만 0.3으로 고치면 OR 논리 연산이 될 수 있다

x_1	x_2	x_1 AND x_2
0	0	0
0	1	0
1	0	0
1	1	1

AND 논리 연산과 결과

x_1	x_2	x_1 OR x_2
0	0	0
0	1	1
1	0	1
1	1	1

OR 논리 연산과 결과

OR 논리 연산을 퍼셉트론으로 구현하자

- OR 논리 연산

```python
import numpy as np

# 가중치와 임계값을 이용한 OR 연산의 구현
def OR(X):
    W = np.array([.5, .5])     # 가중치 값으로 [.5, .5]
    threshold = .3             # 임계값을 .3으로 수정
    sigma = np.sum(X * W)
    if sigma >= threshold:     # 반환값 0, 1을 결정
        return 1
    else:
        return 0

print('OR 논리 연산')
inputs = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
for t in inputs:
    print(t, ':', OR(t))
```

```
OR 논리 연산
[0 0] : 0
[0 1] : 1
[1 0] : 1
[1 1] : 1
```

OR 논리 연산을 퍼셉트론으로 구현하자

- OR 논리 연산의 입력과 출력과정

• 가중치 ω_1, ω_2는 각각 .5, .5로, 임계값은 .3으로 가정함

입력 x_1	입력 x_2	$\sum \omega_i x_i$: 신호의 합 σ	$\phi()$ 함수 : $\sigma \geq .3$	출력
0	0	0×.5 + 0×.5 = .0	거짓(.0 ≥ .3)	0
0	1	0×.5 + 1×.5 = .5	참(.5 ≥ .3)	1
1	0	1×.5 + 0×.5 = .5	참(.5 ≥ .3)	1
1	1	1×.5 + 1×2 = 1.0	참(1.0 ≥ .3)	1

x_1	x_2	x_1 OR x_2
0	0	0
0	1	1
1	0	1
1	1	1

OR 논리 연산과 결과

OR 논리 연산을 퍼셉트론으로 구현하자

도전 문제 5.3: NAND 논리 연산 구현하기

상 중 하



1. NAND 논리 연산은 AND 논리 연산의 결과에 NOT을 취한 것이다. 따라서 다음과 같은 결과를 출력한다. 적절한 가중치와 임계값을 설정하여 NAND 논리 연산을 구현해 보자.

x_1	x_2	x_1 NAND x_2
0	0	1
0	1	1
1	0	1
1	1	0

NAND 논리 연산과 결과

2. NOR 논리 연산은 OR 논리 연산의 결과에 NOT을 취한 것이다. 따라서 다음과 같은 결과를 출력한다. 적절한 가중치와 임계값을 설정하여 NOR 논리 연산을 구현해 보자.

x_1	x_2	x_1 NOR x_2
0	0	1
0	1	0
1	0	0
1	1	0

NOR 논리 연산과 결과

편향값을 추가하자

- 위의 두 코드에서 나타난 함수 AND()와 OR()은 임계값 threshold가 .8, .3이라는 점을 제외하면 모든 부분이 동일
  - 따라서 이 두 가지 논리 연산의 차이점은 오직 임계값만 다르다는 점일 것이다.
  - 이러한 이유로 다음과 같이 좀 더 범용적인 함수인 gate()를 만들 수 있을 것이다.

```python
import numpy as np

# 가중치와 임계값을 이용한 범용 논리 연산의 구현
def gate(X, threshold):          # threshold를 매개변수로 받는 함수
    W = np.array([.5, .5])       # 가중치 값으로 [.5, .5]
    sigma = np.sum(X * W)
    if sigma >= threshold:       # 반환값 0, 1을 결정
        return 1
    else:
        return 0

inputs = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
print('AND 논리 연산')
for t in inputs:
    print(t, ':', gate(t, .8)) # 임계값을 인자 .8로 넘겨줌
```

편향값을 추가하자

- 하나의 함수로 AND와 OR 논리 연산을 구현하였다.
  - threshold 매개변수의 전달인자가 .8이면 AND, .3이면 OR 논리 연산이 구현됨

```
print('OR 논리 연산')
for t in inputs:
    print(t, ':', gate(t, .3)) # 임계값을 인자 .3으로 넘겨줌
```

```
AND 논리 연산
[0 0] : 0
[0 1] : 0
[1 0] : 0
[1 1] : 1
OR 논리 연산
[0 0] : 0
[0 1] : 1
[1 0] : 1
[1 1] : 1
```

x_1	x_2	x_1 AND x_2
0	0	0
0	1	0
1	0	0
1	1	1

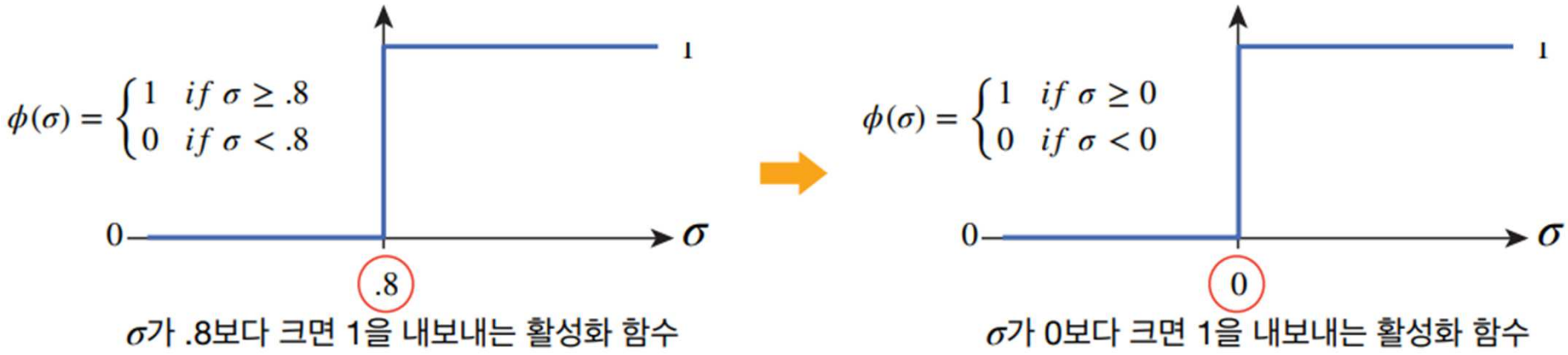
AND 논리 연산과 결과

x_1	x_2	x_1 OR x_2
0	0	0
0	1	1
1	0	1
1	1	1

OR 논리 연산과 결과

편향값을 추가하여 활성화 함수를 간단하게 만들자

- 수식의 치환
  - AND 논리 연산을 구현하는데 사용된 수식 $\sum \omega_i x_i \geq .8$은 $\sum \omega_i x_i - .8 \geq 0$으로
  - OR 논리 연산을 구현하는데 사용된 수식 $\sum \omega_i x_i \geq .3$은 $\sum \omega_i x_i - .3 \geq 0$으로



σ가 .8보다 크면 1을 내보내는 활성화 함수

σ가 0보다 크면 1을 내보내는 활성화 함수

편향값을 추가하여 활성화 함수를 간단하게 만들자

- 오른쪽 변의 식은 0보다 큰지 작은지를 기준으로 삼아 참, 거짓으로 판별식을 만들 수 있다. 두 가지 방법은 내용이 동일하지만 표현식이 조금 다른데, 이와 같이 ≥ 0으로 표현할 경우 다음과 같은 장점이 있다.

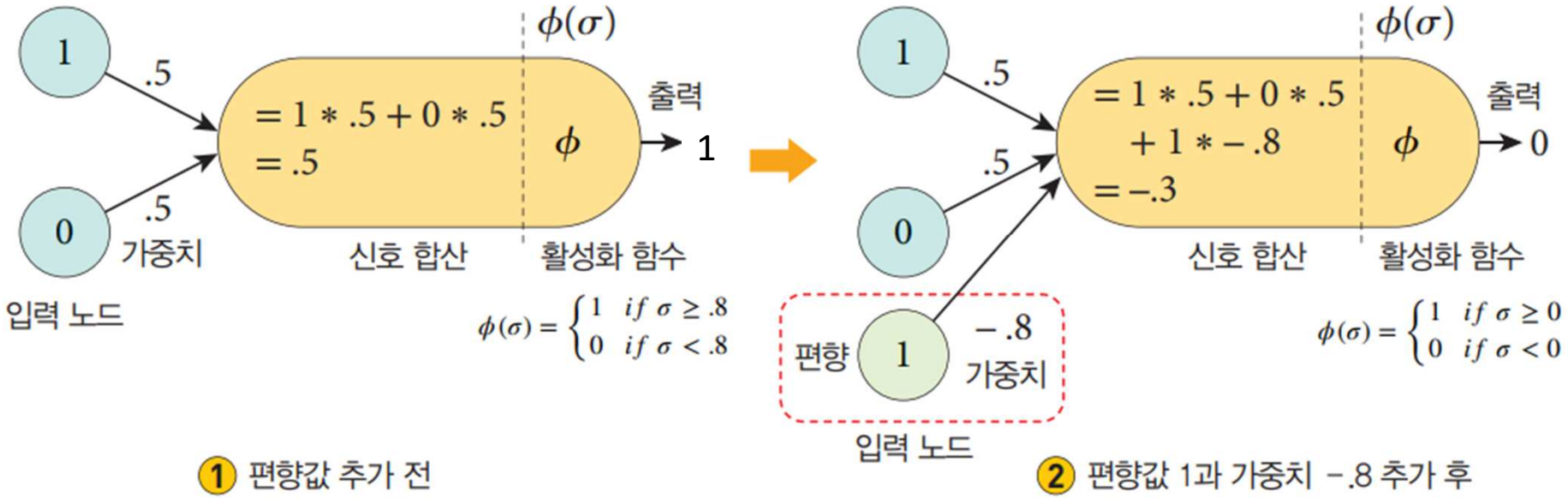
1. 활성화 함수가 간단해진다.
2. 신호의 합을 구하는 코드가 간단해진다.

- 코드에서 사용된 –.8이나 –.3과 같은 값을 앞으로 **편향값** bias 이라고 부를 것이다.
  - 이 값은 입력값이 1인 노드에 대한 가중치로 해석해도 되는데, 편향값이 추가될 경우 활성화 함수는 다음 식과 같다

$$\phi(\sigma) = \begin{cases} 1 & if\ \sigma \geq 0 \\ 0 & if\ \sigma < 0 \end{cases}$$

편향값을 추가하여 활성화 함수를 간단하게 만들자

- 편향값과 활성화 함수
  - 편향값(노드)이 추가되어 활성화 함수의 역할이 간단해짐
  - 0보다 큰가 작은가가 출력 1/0을 결정함



편향값을 가진 퍼셉트론으로 구현하자

- 1행 2열 크기의 [0, 0] 다차원 배열에 1을 추가해서 1행 3열 크기의 [0, 0, 1] 다차원 배열을 만드는 방법을 알아보자.
  - 이를 위해서 원래 다차원 배열원소 t[0]과 t[1]에 새로운 입력값 1을 추가한 배열 x를 만들도록 하자.
  - np.array([t[0], t[1], 1])을 이용하여 입력값 1을 추가

```
inputs = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])

for t in inputs:
    x = np.array([t[0], t[1], 1]) # 노드 1을 추가한 다차원 배열
    print('x =', x)
```

```
x = [0 0 1]
x = [0 1 1]
x = [1 0 1]
x = [1 1 1]
```

편향값을 가진 퍼셉트론으로 구현하자

- 다음으로 activation(s)라는 함수를 만들도록 하자.
  - 이 함수는 입력이 0 이상이면 1을 반환하며, 0보다 작으면 0을 반환한다.
  - 이 함수에 –.3을 입력으로 넣을 경우, 아래와 같이 0을 출력한다.
  - 입력이 .3이면 1을 출력한다

```
import numpy as np

def activation(s):          # s가 0 이상이면 1을 반환, 그렇지 않으면 0을 반환
    return 1 if s >= 0 else 0

print(activation(-.3))      # s가 0 이하의 값
print(activation(.3))       # s가 0 이상의 값
```

```
0
1
```

편향값을 가진 퍼셉트론으로 구현하자

- AND 연산

```python
# 가중치 값에 따라 다른 값을 반환하는 범용 논리 연산
def perceptron(X, W):
    sigma = np.sum(X * W)
    return activation(sigma)          # 활성화 함수에 sigma 값을 넘겨주자

def AND(x):
    X = np.array([x[0], x[1], 1])     # x에 노드 1을 새로 추가
    W = np.array([.5, .5, -.8])       # 가중치 값 설정하기
    return perceptron(X, W)

inputs = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
print('AND 논리 연산')
for t in inputs:
    print(t, ':', AND(t))             # 입력값을 넘겨줌
```

```
AND 논리 연산
[0 0] : 0
[0 1] : 0
[1 0] : 0
[1 1] : 1
```

편향값을 가진 퍼셉트론으로 구현하자

- OR, NAND 연산도 쉽게 구현
  - 가중치 노드의 값을 고치면 됨
- AND : [.5, .5, -.8]
- OR : [.5, .5, -.3]
- NAND : -[.5, .5, -.8]
  - AND 가중치에 음의 부호

```python
def OR(x):
    X = np.array([x[0], x[1], 1]) # X에 1 노드를 추가
    W = np.array([.5, .5, -.3])   # 가중치 값 설정하기
    return perceptron(X, W)

def NAND(x):
    X = np.array([x[0], x[1], 1]) # X에 1 노드를 추가
    W = np.array([.5, .5, -.8])   # 가중치 값 설정하기
    return perceptron(X, -W)      # 가중치 W의 부호를 음수로 고침

inputs = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
print('OR 논리 연산')
for t in inputs:
    print(t, ':', OR(t))      # 입력값과 가중치를 넘겨줌

print('NAND 논리 연산')
for t in inputs:
    print(t, ':', NAND(t))    # 입력값과 가중치를 넘겨줌
```

```
OR 논리 연산
[0 0] : 0
[0 1] : 1
[1 0] : 1
[1 1] : 1
NAND 논리 연산
[0 0] : 1
[0 1] : 1
[1 0] : 1
[1 1] : 0
```

편향값을 가진 퍼셉트론으로 구현하자

도전 문제 5.4: NOR 논리 연산 구현하기

상 중 하

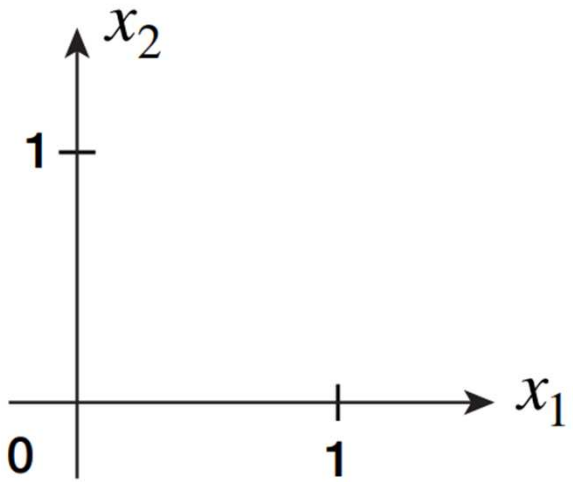


1. NOR 논리 연산은 OR 논리 연산의 결과에 NOT을 취한 것이다. 위의 코드를 수정하여 NOR 논리 연산으로 구현해 보도록 하자. 출력은 다음과 같이 나타나도록 한다.

   힌트 OR 논리 연산의 가중치 W의 부호를 −W와 같이 음수로 반전시켜 보자.

```
NOR 논리 연산
[0 0] : 1
[0 1] : 0
[1 0] : 0
[1 1] : 0
```

- 논리 연산의 결과를 시각화하자
  - 앞 절에서 다룬 논리 연산의 결과를 평면 공간에서 확인하는 방법을 알아보자.
  - 우선 AND 연산의 입력 x_1과 x_2를 그림과 같이 직교하는 직선으로 두고 평면 공간에 배치시키자.
  - 입력값 x_1은 수평축에 두고, x_2는 수직축에 두었다. 이 그림에서는 $\phi()$ 함수 대신에 y라는 이름의 변수를 만들어서 이 변수가 0이 되는 곳은 흰색(거짓 값을 가지는 경우), 1이 되는 곳은 연한 노란색(참 값을 가지는 경우) 영역으로 표시하려고 한다.

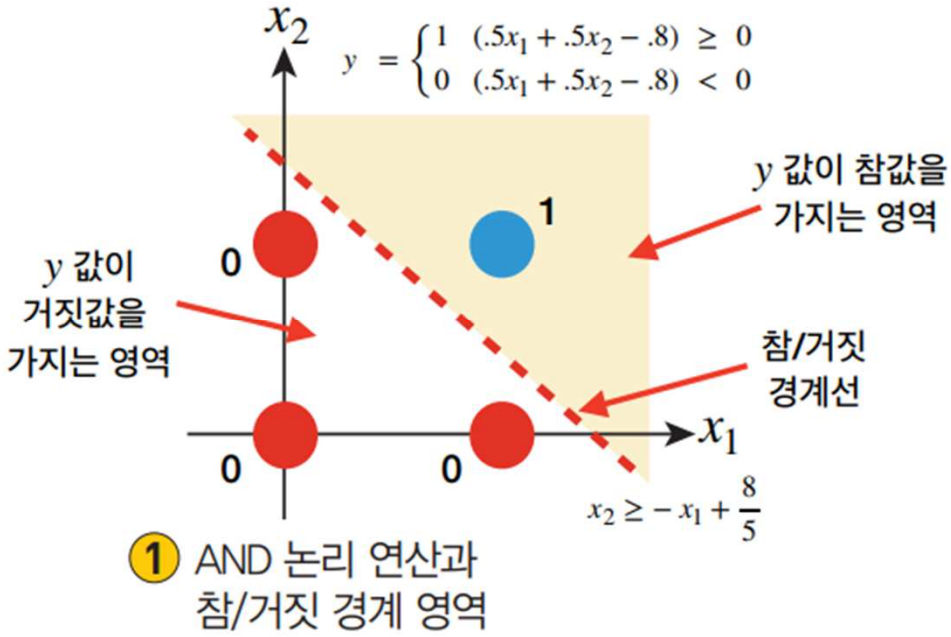


두 변수를 가지는 평면

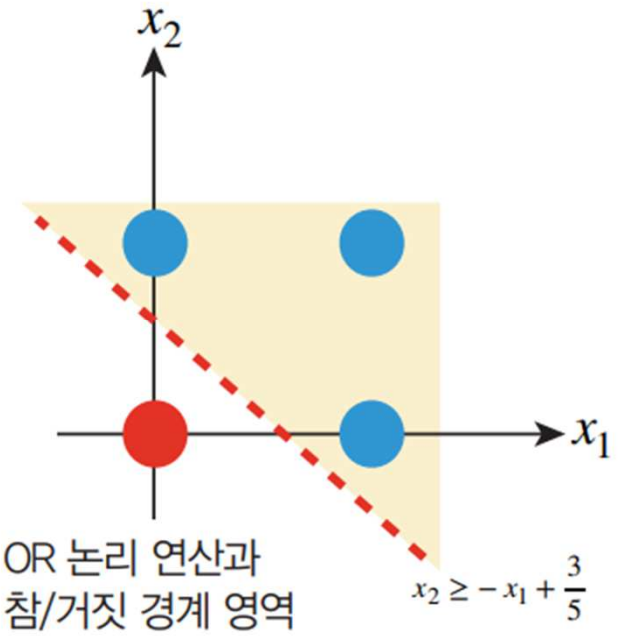


논리 연산의 결과를 시각화하자

- AND 논리 연산의 가중치 매개변수는 (.5, .5, –.8)이므로 이 가중치를 이용하여 $.5x_1 + .5x_2 - .8 \geq 0$과 같은 식을 만들 수 있다.
  - 이 식을 전개하면 $x_2 \geq -x_1 + 8/5$ 이라는 식을 만들 수 있는데, 이 식이 참이 되는 조건은 기울기가 –1이고 x_2 절편이 8/5 인 직선에서 직선의 위쪽에 있는 모든 점들의 모임이다.



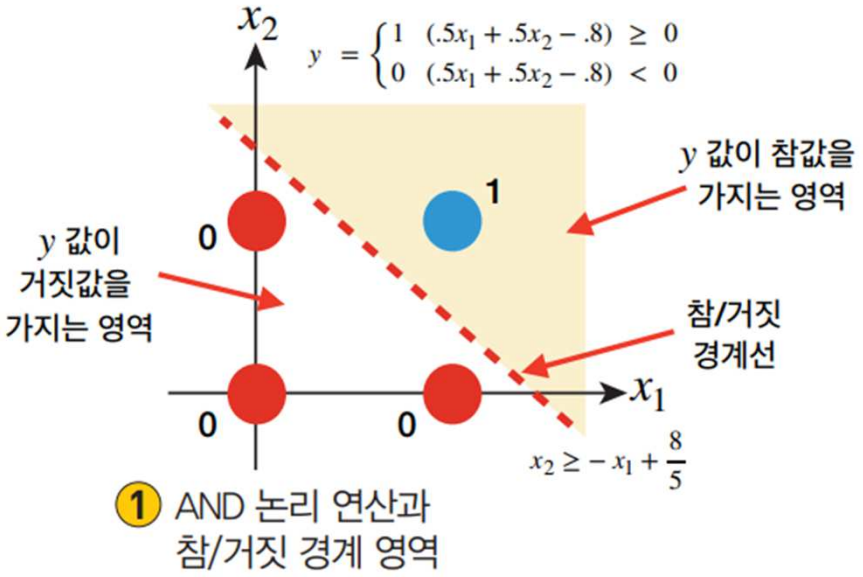
① AND 논리 연산과 참/거짓 경계 영역



② OR 논리 연산과 참/거짓 경계 영역

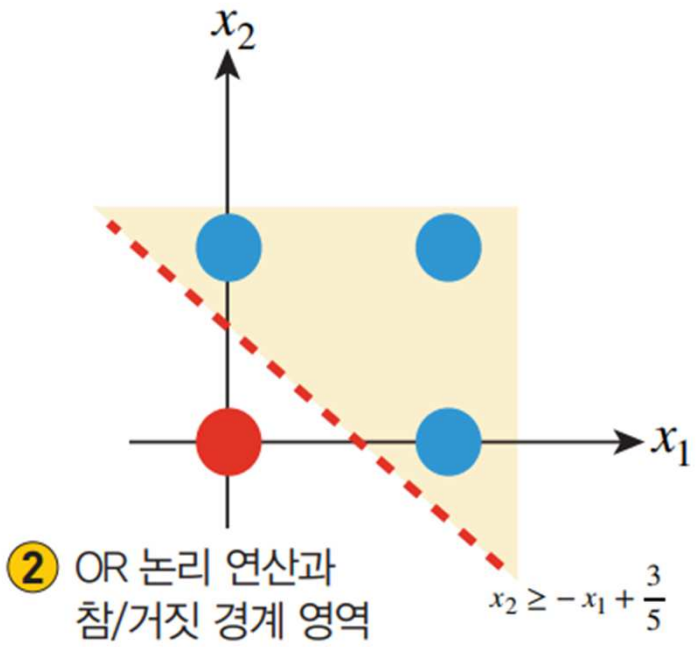
논리 연산의 결과를 시각화하자

- 그림 1 에 표시된 네 개의 점은 x_1과 x_2 값이 0과 0이 되는 경우, 0과 1이 되는 경우, 1과 0이 되는 경우, 1과 1인 경우이다.
  - 이 점들 중에서 x_1과 x_2 값이 0과 0이 되는 경우 출력은 0이 되며, 이는 빨간색 점으로 표시되어 있다.
- 반면 x_1과 x_2 값이 1과 1이 되는 경우에는 출력이 1이 되는데 이 점은 파란색 점으로 표시되어 있다. 이 평면을 나누는 경계 직선은 빨간색 점선으로 나타나 있다



논리 연산의 결과를 시각화하자

- OR 논리 연산과 참/거짓 경계 영역의 의미를 설명하는 것은 그림 2 인데 경계선의 식이 $x_2 \geq -x_1 + 3/5$ 이므로 y 값이 참이 되는 점은 x_1과 x_2 값이 0과 1이 되는 경우, 1과 0이 되는 경우, 1과 1인 경우이다.
  - 결국 우리가 만든 가중치와 변수의 곱 그리고 활성화 함수는 **공간을 나누는 직선을 어떻게 설정하는가의 문제로 치환**할 수 있다



② OR 논리 연산과 참/거짓 경계 영역

논리 연산의 결과를 시각화하자

도전 문제 5.5: OR 논리 연산의 시각화

상 중 하



1. 가중치가 (.5, .5, −.2)인 OR 논리 연산을 시각화하자.

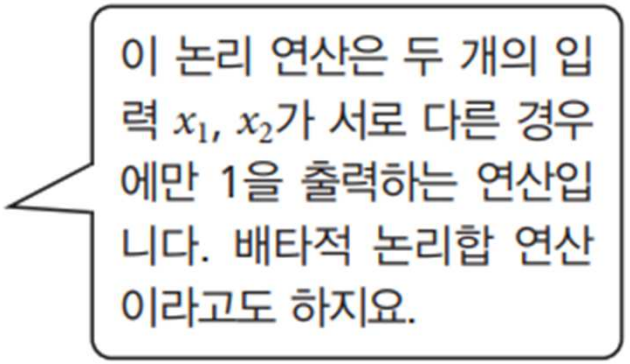
2. 가중치가 (−.5, −.5, .8)인 NAND 논리 연산이 분할하는 공간을 시각화하자.

인공지능 연구의 침체기를 가져온 XOR 문제

- 조금 복잡한 논리 연산을 수행하는 퍼셉트론을 어떻게 구현할 수 있을지 살펴보자.
  - XOR 논리 연산은 두 개의 입력에 대하여 입력값이 서로 다른 경우에만 1을 출력하는 연산이다.
  - 따라서 그림과 같이 0, 1 또는 1, 0으로 서로 다른 경우에만 1이 출력되고 나머지 입력에 대해서는 0을 출력한다.
  - 이러한 특징을 가지는 **XOR 논리 연산**을 **배타적 논리합 연산**이라고 한다.

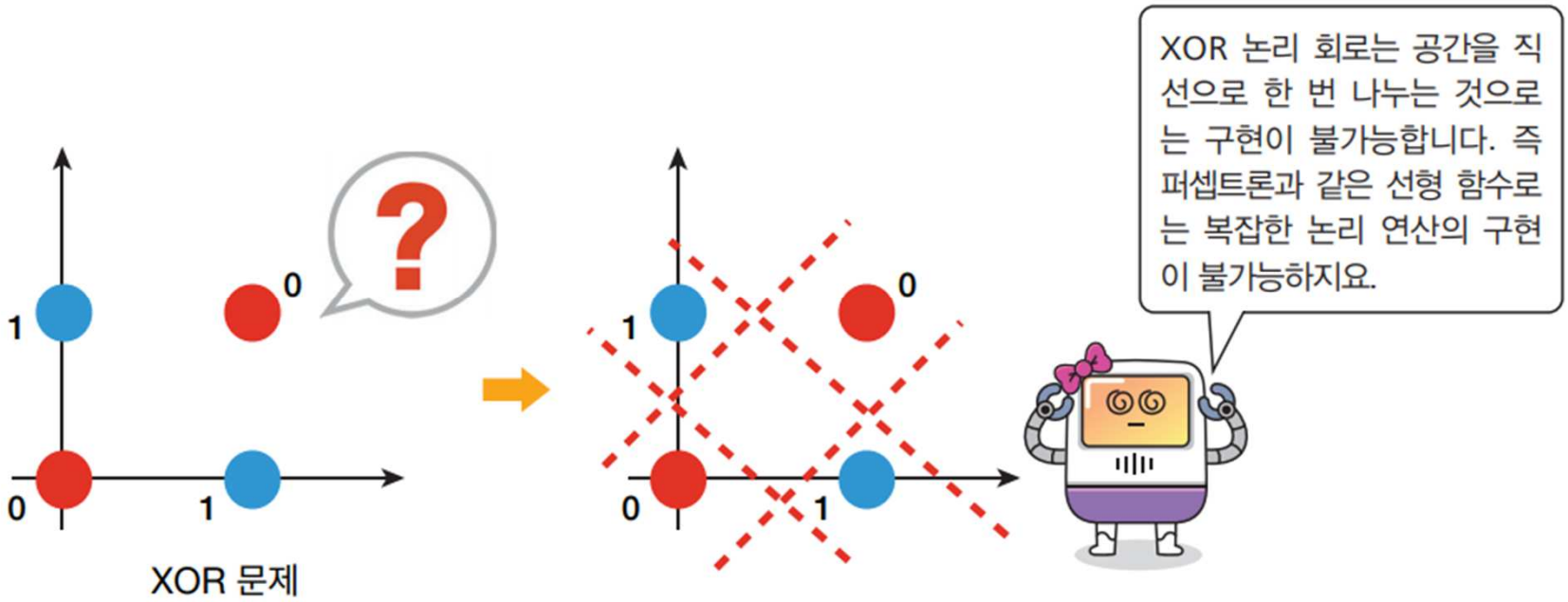
x_1	x_2	x_1 XOR x_2
0	0	0
0	1	1
1	0	1
1	1	0

XOR 논리 연산과 결과



인공지능 연구의 침체기를 가져온 XOR 문제

- 다음의 그림을 살펴보면 평면에 0과 1의 값을 가지는 네 개의 점이 있는데 이들은 어떻게 직선을 긋더라도 한 번에 분리가 불가능하다.
  - 즉, 앞서 살펴본 퍼셉트론과 같은 선형 함수로는 고차원의 복잡한 논리 연산의 구현이 불가능하다.



인공지능 연구의 침체기를 가져온 XOR 문제

- 이 문제를 지적한 사람은 **마빈 민스키**Marvin Lee Minsky라는 학자
  - 1969년에 집필한 《퍼셉트론》이라는 저서를 통해서 퍼셉트론이 XOR 연산을 구현하지 못한다는 것을 주장하였다.
  - 마빈 민스키 교수는 MIT 인공지능 연구소의 공동 설립자이며 1970년에 튜링상을 받은 인공지능의 권위자이다.
  - 그는 퍼셉트론 연구에 매우 회의적이었으며 "XOR 연산조차 해내지 못하는 기계가 어떻게 지능을 구현할 수 있을까"라는 의구심을 가지고 있었다.
  - 그의 저서는 퍼셉트론 연구에 매우 부정적인 영향을 가져왔고 인공지능 연구비 축소와 연구자의 이탈이라는 결과를 낳게 되었다.
  - 결국 이는 신경망 분야의 긴 침체를 가져올 수밖에 없었다.
  - 이 침체기를 **1차 인공지능의 겨울**이라 함



인공지능의겨울

인공지능 연구의 침체기를 가져온 XOR 문제

- 선형 함수와 비선형 함수를 알아보자

NOTE: 선형 함수 vs 비선형 함수



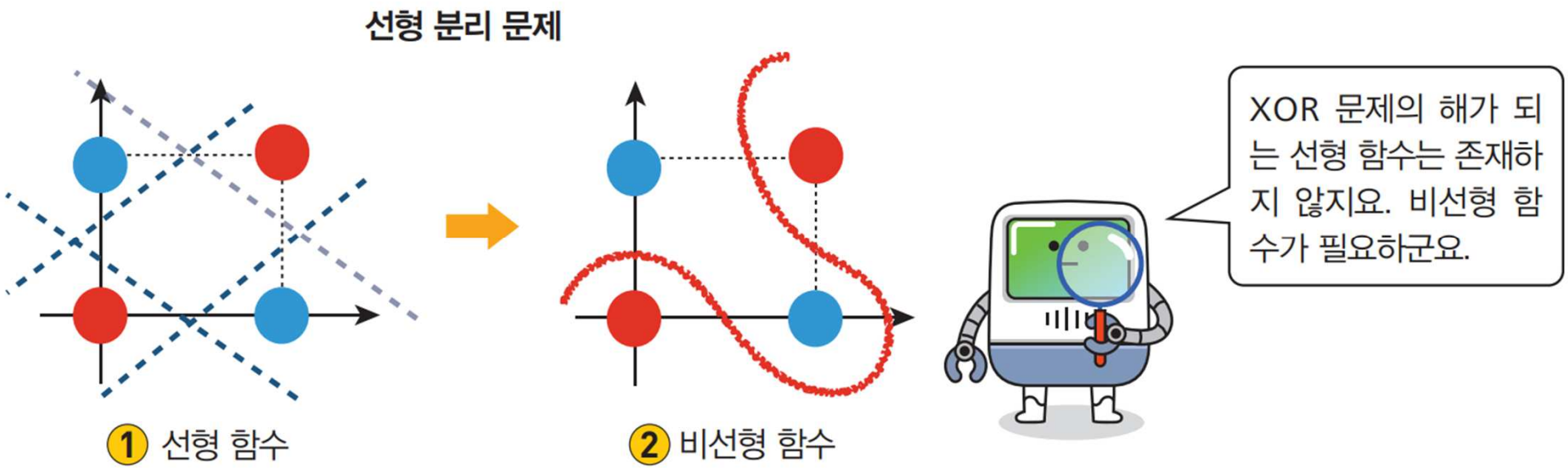
선형 함수는 그래프를 그렸을 때 직선의 형태로 나타나는 함수를 말한다. 이 함수는 어떤 입력에 대해서도 결과가 항상 일정 비율로 증가하거나 감소한다는 것을 의미한다. 예를 들어, $y = 2x$라는 함수는 선형 함수이다. 이 함수의 입력 x가 1일 때 y는 2, x가 2일 때 y는 4로, x가 증가하면 y도 일정 비율로 증가한다.

비선형 함수는 그래프를 그렸을 때 직선의 형태로 나타나지 않는 함수이다. 이것은 어떤 입력에 대해서도 결과가 일정한 비율로 증가하거나 감소하지 않는다는 것을 의미한다. 예를 들어, $y = x^2$이라는 함수는 비선형 함수이다. 이 함수에서 x가 1일 때 y는 1, x가 2일 때 y는 4로, x가 일정한 비율로 증가하더라도 y는 동일한 비율로 증가하지 않는다.

이처럼 선형 함수는 일정한 비율로 변화하는 반면, 비선형 함수는 변화가 일정하지 않으며 여러 방향으로 복잡하게 변화할 수 있다. 따라서 비선형 함수를 사용하면 복잡한 문제를 더 잘 해결할 수 있다.

은닉 노드를 가진 다층 퍼셉트론의 등장

- 선형 함수 만으로는 XOR 문제를 해결할 수 없음!!
  - 여러 개의 층을 만들면 가능함 : 은닉 노드를 가진 **은닉층**hidden layer이 필요

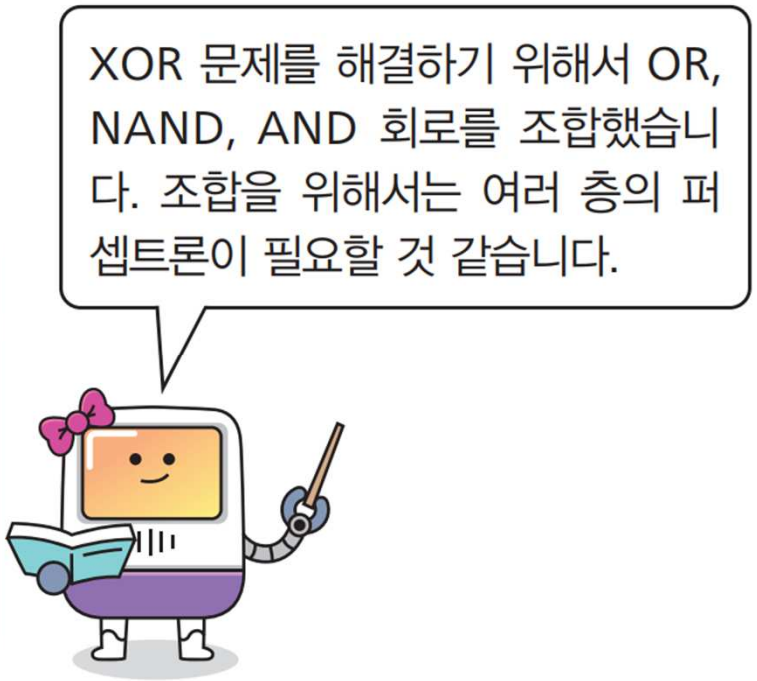


은닉 노드를 가진 다층 퍼셉트론의 등장

- 다음과 같이 OR, NAND, AND 회로를 조합하자
  - XOR 논리 회로를 만들 수 있다.

		A	B	
x_1	x_2	x_1 OR x_2	x_1 NAND x_2	A AND B
0	0	0	1	0
0	1	1	1	1
1	0	1	1	1
1	1	1	0	0

XOR



은닉 노드를 가진 다층 퍼셉트론의 등장

- 코드로 구현

```python
import numpy as np

def activation(s):
    return 1 if s >= 0 else 0

# 가중치 값에 따라 다른 값을 반환하는 범용 논리 연산
def perceptron(X, W):
    sigma = np.sum(X * W)
    return activation(sigma)          # 활성화 함수에 sig

def AND(x):
    X = np.array([x[0], x[1], 1])     # x에 1 노드를 추가
    W = np.array([.5, .5, -.8])       # 가중치 값 설정하기
    return perceptron(X, W)

def OR(x):
    X = np.array([x[0], x[1], 1])     # X에 1 노드를 추가
    W = np.array([.5, .5, -.3])       # 가중치 값 설정하기
    return perceptron(X, W)

def NAND(x):
    X = np.array([x[0], x[1], 1])     # X에 1 노드를 추가
    W = np.array([.5, .5, -.8])       # 가중치 값 설정하기
    return perceptron(X, -W)          # 가중치 W의 부호를 음수로

def XOR(x):                # 여러 게이트의 조합으로 구현된 XOR 게이트
    return AND([OR(x), NAND(x)])
```

```python
print('--- 다층 퍼셉트론으로 구현한 XOR 논리 연산 ---')
inputs = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
for x in inputs:
    y = XOR(x)
    print(x, ':', y)
```

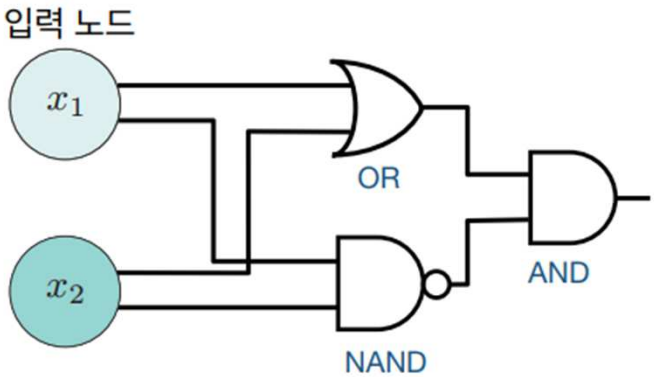
```
--- 다층 퍼셉트론으로 구현한 XOR 논리 연산 ---
[0 0] : 0
[0 1] : 1
[1 0] : 1
[1 1] : 0
```

X	Y	출력
0	0	0
0	1	1
1	0	1
1	1	0

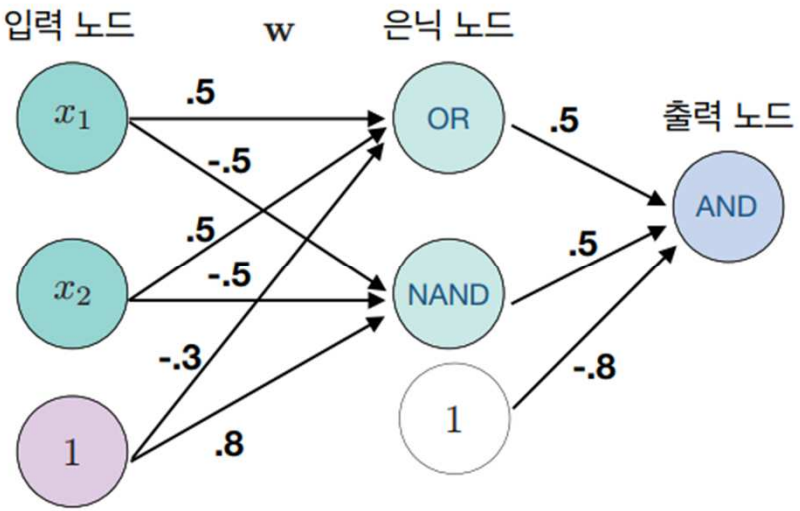
XOR 논리 연산

은닉 노드를 가진 다층 퍼셉트론의 등장

- 입력 노드, 은닉 노드, 출력 노드를 가진 그래프 형태로 그려 보자.
  - 왼쪽 그림은 OR, NAND, AND 논리 연산을 조합하여 구현한 XOR 연산이며 오른쪽 그림은 다층 퍼셉트론으로 구현한 XOR 논리 연산이다.
  - 구현은 가능하지만 어떻게 적절한 가중치 값을 학습시킬 것인가에 대해서는 큰 문제를 안고 있다.



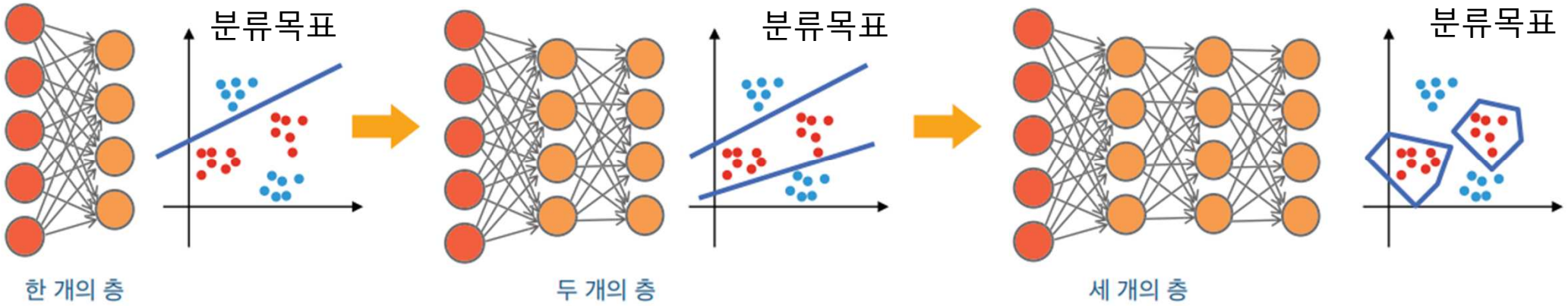
OR, NAND, AND 논리 회로를 조합하여 구현한 XOR 논리 회로



XOR 연산을 수행하는 다층 퍼셉트론과 가중치

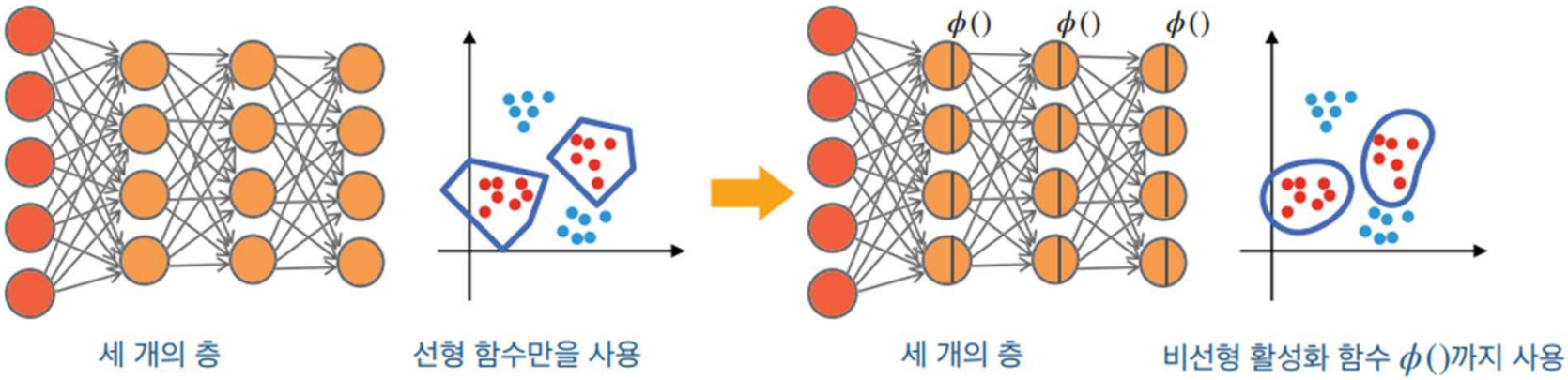
다층 퍼셉트론으로 해결할 수 있는 문제들

- 퍼셉트론의 층이 증가하면 할수록 모델이 자유롭게 움직일 수 있는 가능성이 증가한다.
  - 이와 같이 모델이 자유롭게 움직이는 현상을 **자유도**라고 한다.
  - 다음 그림을 살펴보면 층이 증가하면 할수록 오른쪽 그룹에 대한 분류가 더 세분화되는 것을 볼 수 있다.



다층 퍼셉트론으로 해결할 수 있는 문제들

- 두 개의 그룹을 나누는 예시
  - 왼쪽은 직선만을 사용(선형 함수)하여 그룹을 나눔.
  - 오른쪽은 **비선형 활성화 함수** nonlinear activation function 를 사용하기 때문에 곡선 형태의 모양까지 만들 수 있다.



- 깊은 층을 가진 퍼셉트론 모델은 다차원의 복잡한 모델이기 때문에 곡선이 아닌 **초평면** hyperplane 이 될 수 있다.
  - 초평면이란 단어는 고차원의 공간에서 사용되는 용어로, 2차원의 직선과 3차원의 평면을 더 높은 차원으로 확장시킨 평면이다

다층 퍼셉트론으로 해결할 수 있는 문제들

도전 문제 5.6: 많은 층을 가진 퍼셉트론

상 중 하

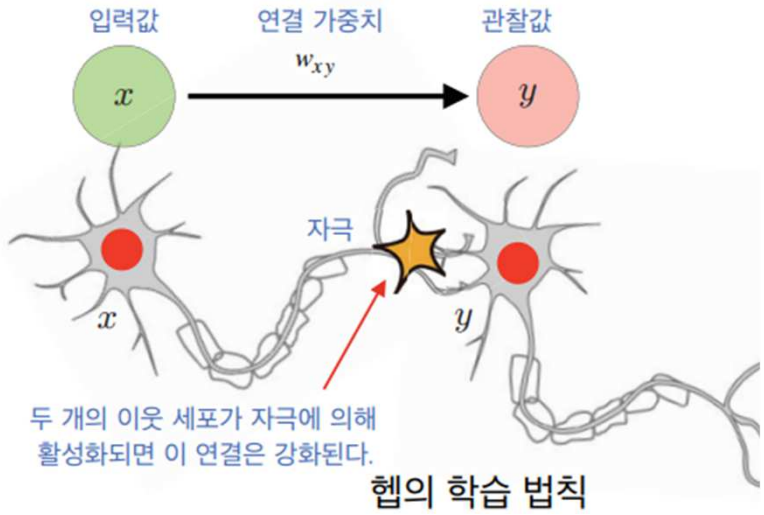


1. 퍼셉트론이 많은 층을 가질수록 얻게 되는 장점들과 단점들을 나열해 보자.

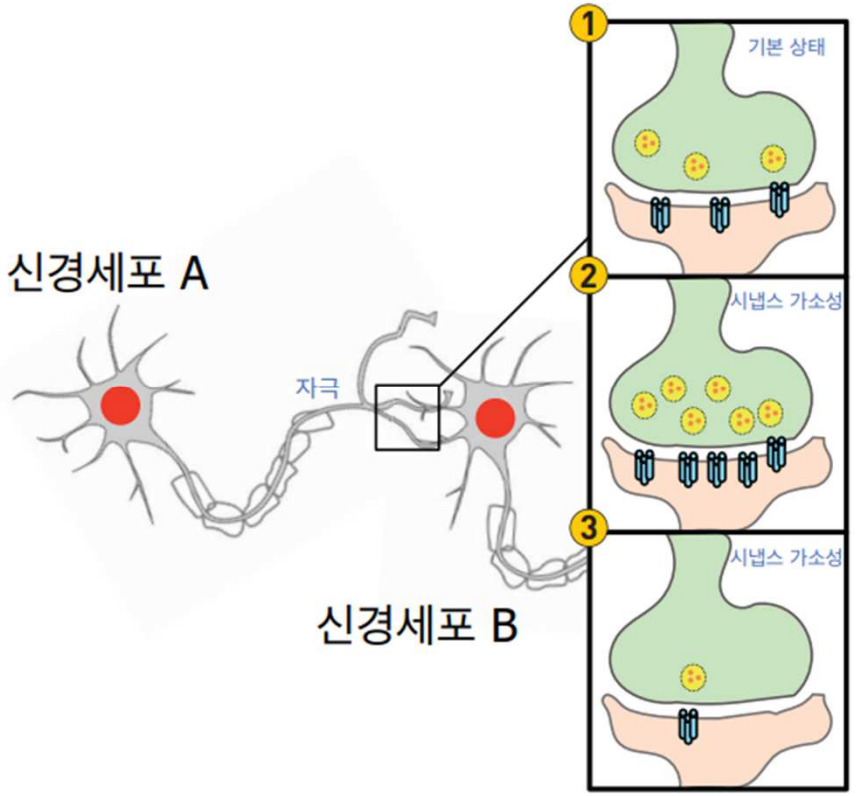
2. 활성화 함수로 비선형 함수를 사용하는 이유를 기술해 보자.

04 퍼셉트론 학습시키기

- 퍼셉트론과 학습의 원리
  - 가중치와 편향을 사용자가 지정하지 않고 학습을 통해서 얻도록 하자.
  - 헵의 학습 법칙을 이해하자.



헵의 학습 법칙



이웃 신경세포로부터 전달되는 자극이 증가하면 발화율도 증가한다. 즉, 함께 활성화되는 세포는 함께 연결된다고 볼 수 있다.

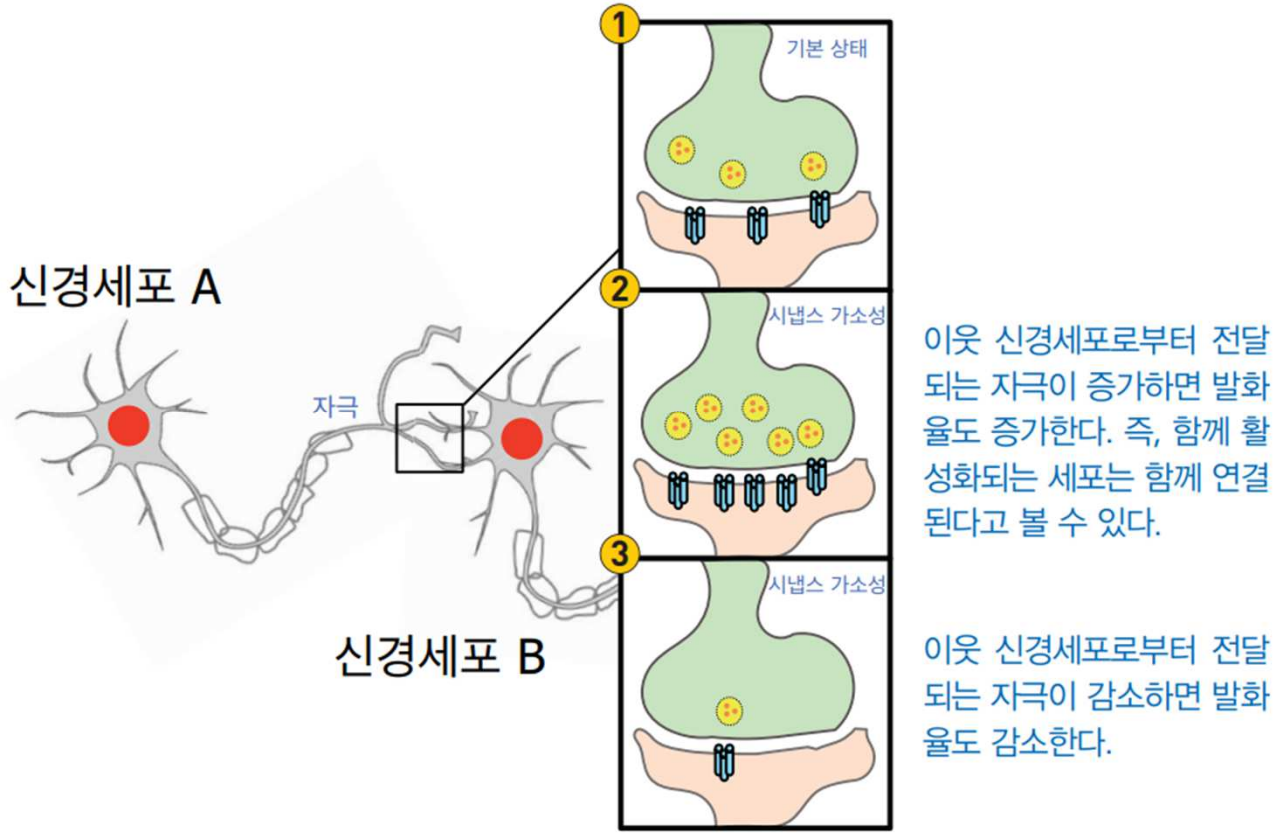
이웃 신경세포로부터 전달되는 자극이 감소하면 발화율도 감소한다.

퍼셉트론과 학습의 원리

- 인간과 동물의 뇌 신경세포는 학습을 하거나 자극을 받을 때 일정한 물리적, 구조적 변화가 일어난다.
  - 새로운 사실을 배울 때마다 뇌는 그 구조가 조금씩 변하고 경험과 학습을 통해 수시로 변화를 겪는 특징이 있다.
  - 캐나다의 신경 심리학자인 도널드 헵Donald Hebb은 "신경세포 A의 축삭돌기가 다른 신경세포 B를 자극하기에 충분할 정도로 가까이 있으면서 이 세포를 활성화하는 일에 반복적이거나 지속적으로 참여한다면, 두 세포 가운데 하나나 둘 모두에서 성장이나 대사 변화가 발생하여 B를 활성화하는 세포로서 A가 가진 효능이 증가한다"라고 하였다.

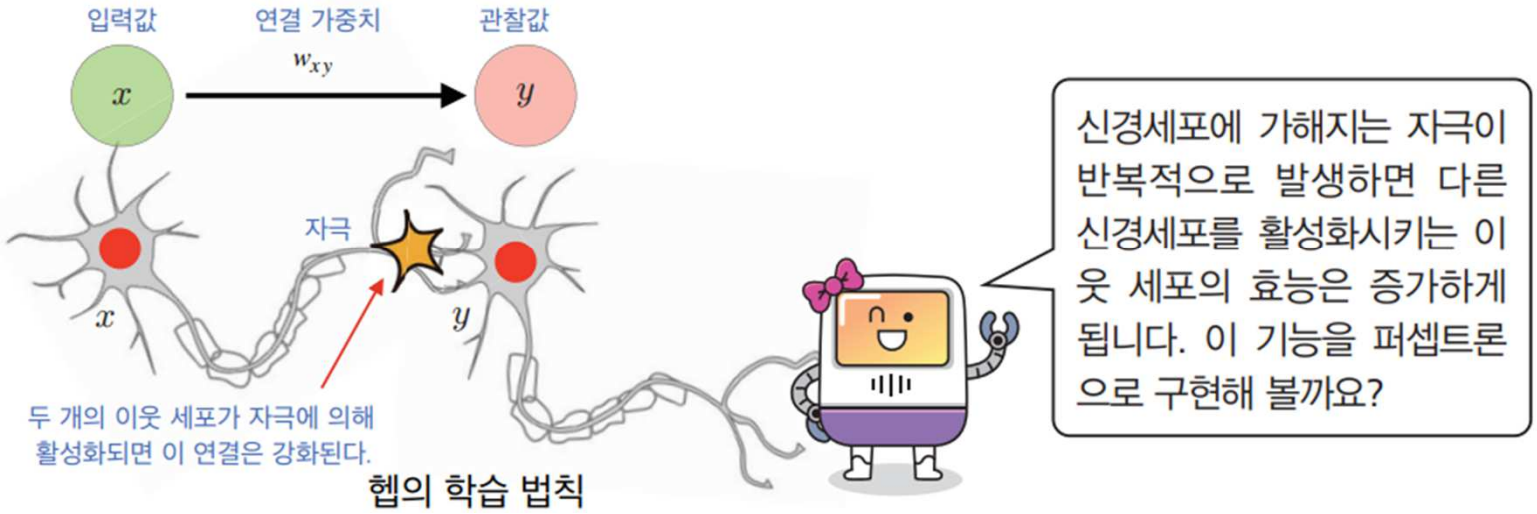
퍼셉트론과 학습의 원리

- "함께 활성화되는 세포는 함께 연결된다"라는 표현을 알아보자
  - 그림의 1 은 신경세포 A와 인접한 신경세포 B의 기본 상태이다.
    - 신경세포 A로부터 전달되는 자극은 다양한 신경전달 물질의 형태로 신경세포 B에 전달된다.



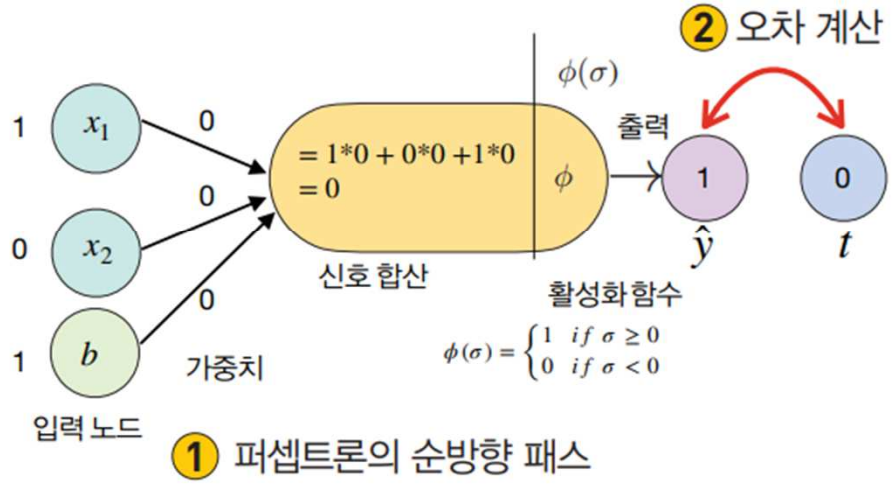
퍼셉트론과 학습의 원리

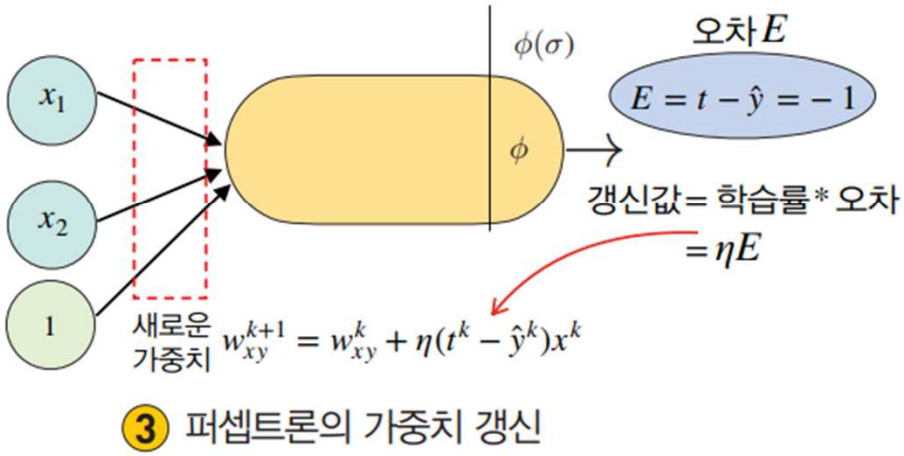
- x는 입력값을, y는 추정값 또는 출력값을, ω는 연결 가중치를 의미한다.
  - 그리고 x^k, y^k, ω^k는 k 시점의 x, y, ω 값이며, ω_{xy}는 x, y 사이의 연결 가중치를 의미한다.
  - 즉, k 시점 연결 가중치(ω_k)를 **동일한 시점의 x, y 값을 바탕으로 갱신하여 새로운 연결 가중치를 구하는 것이 곧 학습**이라고 할 수 있을 것이다.



퍼셉트론과 학습의 원리

- 퍼셉트론에서 학습이란
  - 신경세포가 달성해야 할 **목표치를 제시하고 연결 가중치가 목표치를 만족시키도록 만들어가는 것이 딥러닝의 학습**과정이다



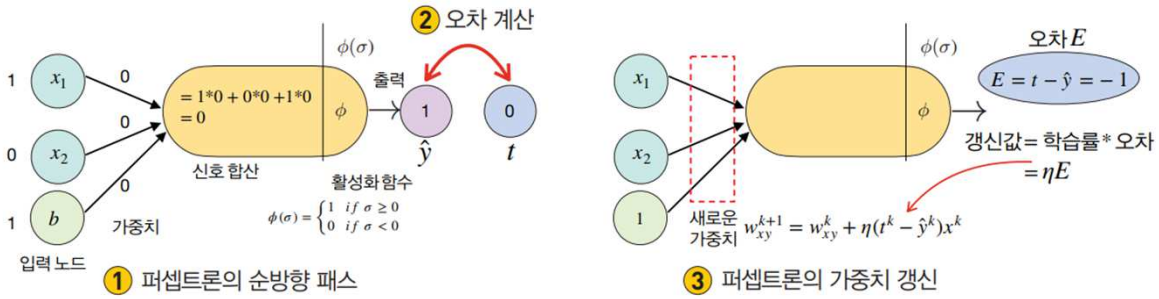


$$\omega_{xy}^{k+1} = \omega_{xy}^{k} + \eta(\mathrm{t}^k - \hat{y}^k)x^k$$

퍼셉트론과 학습의 원리

- 그림의 과정은 한 번에 이루어지지 않는데, 이 신경망의 학습은 시간의 흐름에 따라 가중치가 갱신되는 구조를 가진다.
  - k 시점의 다음 시점인 k + 1 시점 가중치의 갱신은 **학습률** learning rate 을 통해 조금씩 이루어지게 만든다.
  - k 시점의 목표값 t^k와 출력값(또는 추정값) y의 차이 $t^k - \hat{y}^k$가 오차(Ɛ)이다.
  - 일반적으로 목표값 t^k와 t는 동일하다.
  - 학습률은 그리스 문자 η(에타라고 읽음)를 사용하여 표기한다.
  - 이제 퍼셉트론의 학습은 다음과 같은 식으로 표현될 수 있다.

$$\omega_{xy}^{k+1} = \omega_{xy}^{k} + \eta(t^k - \hat{y}^k)x^k$$



- 이 재귀식의 특징

$$\omega_{xy}^{k+1} = \omega_{xy}^{k} + \eta(t^k - \hat{y}^k)x^k$$

1. 가중치 ω_{xy}^{k+1}은 이전 가중치 ω_{xy}^{k}를 바탕으로 갱신된다.
2. 가중치가 갱신되는 정도는 $t^k - \hat{y}^k$로 표현되는 오차에 학습률(η)을 곱한다.
3. 오차와 학습률 그리고 입력 x의 곱과 ω_{xy}^{k}의 합이 최종적으로 갱신되는 새로운 가중치 값이다.

- 방대한 딥러닝의 은닉층에 있는 가중치를 오차와 학습률을 이용하여 갱신하는 것은 여전히 어려운 문제이다 – 오차 역전파로 가능함
- 딥러닝을 위해서는 빠른 병렬처리와 행렬 연산이 꼭 필요하다.

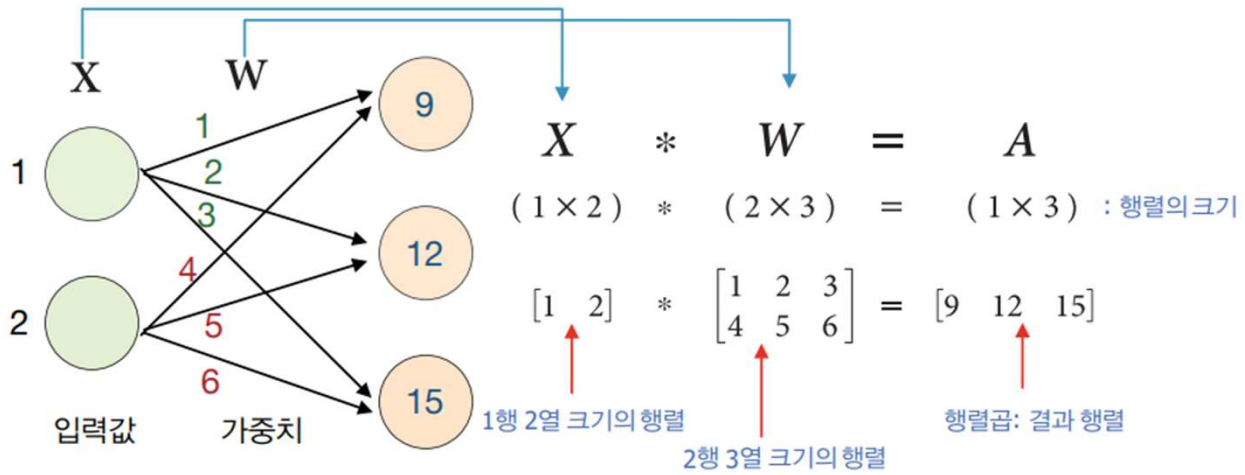
퍼셉트론의 학습을 위한 행렬 표현 방법

- 행렬 표현의 필요성
  - 딥러닝은 많은 수의 퍼셉트론을 한꺼번에 학습시켜야만 하는 어려운 일을 해야 한다.
  - 이러한 일은 행렬을 이용하면 매우 효율적으로 할 수 있다.
  - X 행렬의 크기가 1×2 이고, W 행렬의 크기가 2×3이므로 이 두 행렬의 곱은 1행 3열 (1×3 형태) 크기의 행렬이 된다.
  - 오른쪽 그림의 복잡한 신경망 구조는 행렬을 사용하면 간단한 하나의 식으로 표현이 가능하다

$$\mathbf{X} = [x_1 \quad x_2] = [1 \quad 2]$$

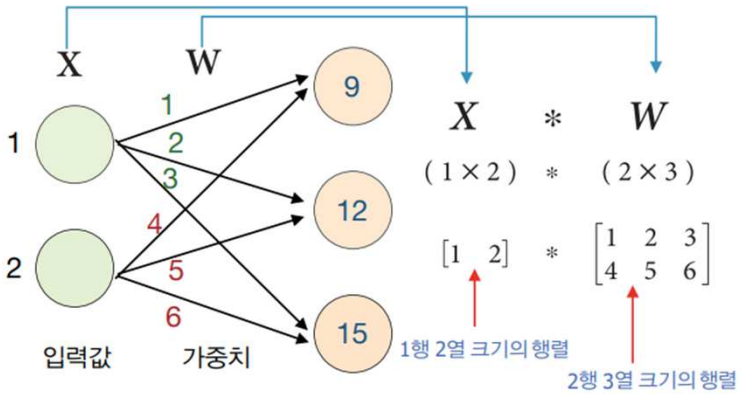
$$\mathbf{W} = \begin{bmatrix} \omega_{11} & \omega_{21} & \omega_{31} \\ \omega_{12} & \omega_{22} & \omega_{32} \end{bmatrix} = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$$

$$\mathbf{A} = \mathbf{XW} = [9 \quad 12 \quad 15]$$



퍼셉트론의 학습을 위한 행렬 표현 방법

- 이 연산을 넘파이로 구현하자.




```
X = np.array([1, 2])                        # 넘파이 ndarray 객체 X 생성
W = np.array([[1, 2, 3], [4, 5, 6]]) # 2x3 크기의 ndarray 객체 W 생성
A = np.dot(X, W)
print('A =', A)
```

```
A = [ 9 12 15]
```

퍼셉트론의 학습을 위한 행렬 표현 방법

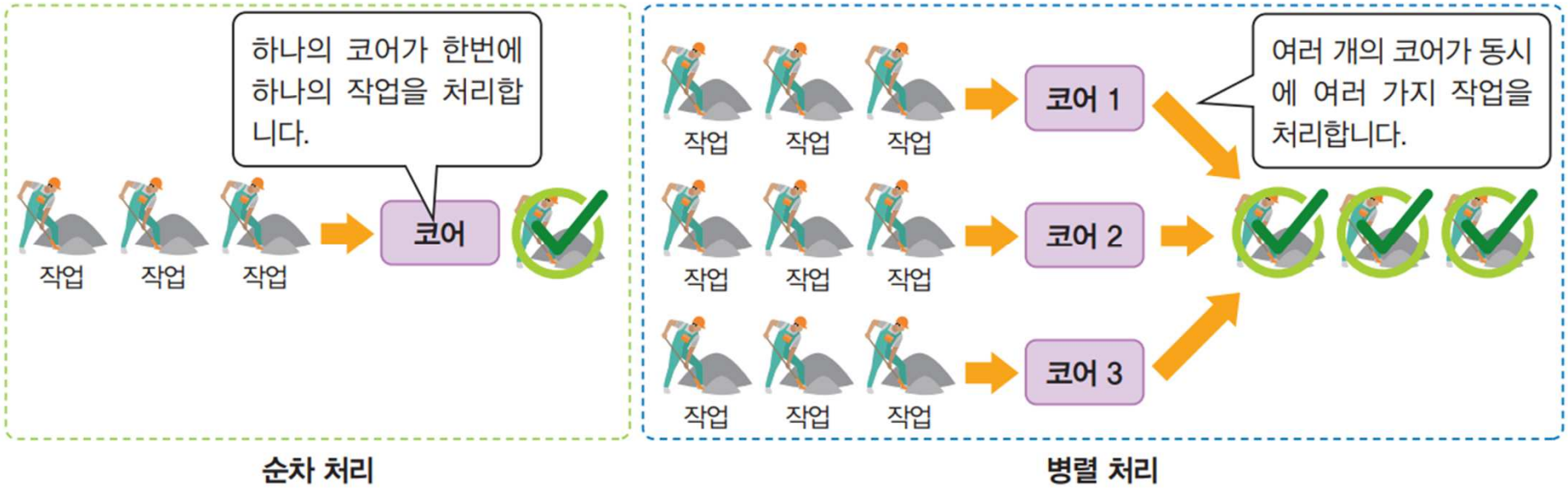
- 이 수식에 편향 행렬 B까지 추가하여 표현할 수 있다.

$$A = XW + B$$

- 이 간단한 식은 딥러닝의 원리를 설명하는 기본적이면서도 중요한 식이다. 이 식의 출력에 해당하는 A 행렬은 다음 층의 입력 X가 되며, 이러한 구조의 반복으로 깊은 층의 신경망을 구현할 수 있다.
  - 입력 값이 100개를 넘어 1,000개, 10,000개인 경우도 가정할 수 있는데 이렇게 방대한 연산을 하기 위해서는 빠른 병렬 처리 기능이 요구된다. 그리고 이를 잘 수행하는 것이 바로 **병렬 처리 시스템**이다

동시에 여러 가지 작업을 처리하는 병렬 처리 시스템

- 순차 처리 시스템 vs 병렬 처리 시스템



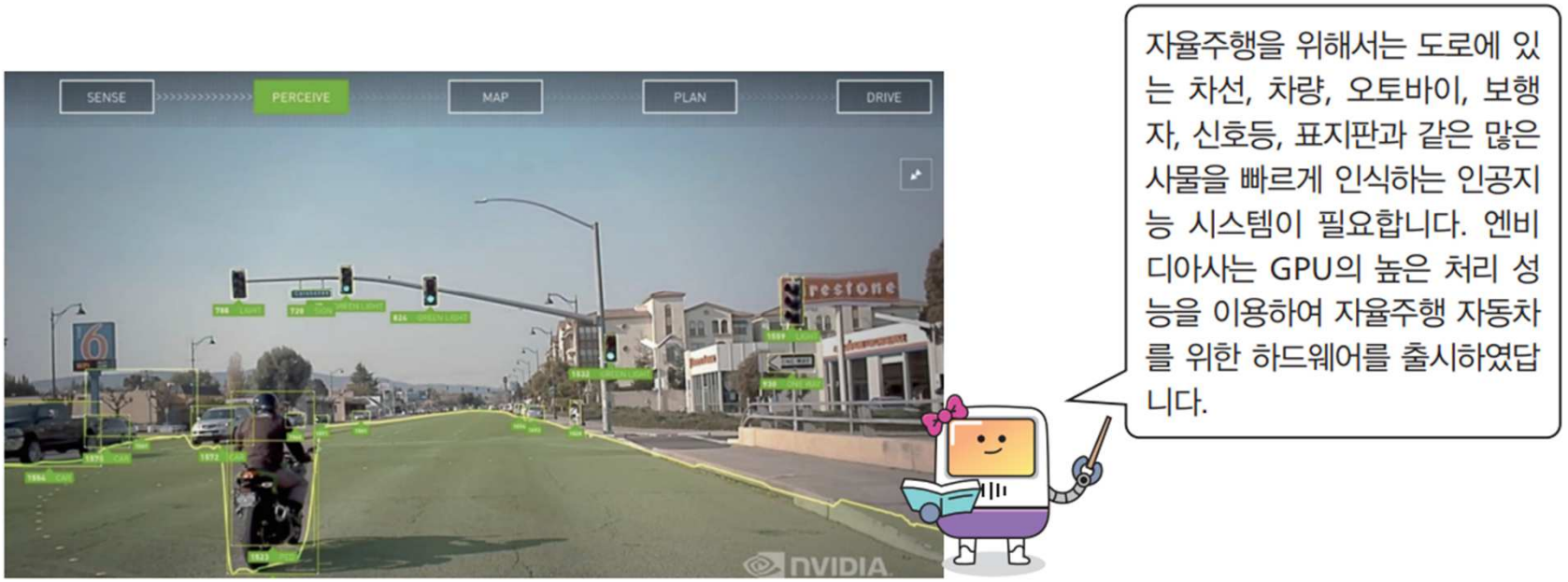
동시에 여러 가지 작업을 처리하는 병렬 처리 시스템

- 엔비디아 NVIDIA 사의 최신 그래픽 카드는 수 천 개에서 만 개 이상의 그래픽 병렬 처리 코어를 가진 그래픽 카드를 매년 출시하여 병렬 처리 작업을 빠르게 하도록 도와준다

	CPU	GPU
주된 목적	컴퓨터의 연산과 제어를 담당하는 핵심 부품	컴퓨터의 그래픽 처리를 담당하는 목적으로 시작되었으며, 최근 인공지능을 위한 병렬 처리까지 효율적으로 수행함
코어의 수	1~8개가량의 적은 코어	수백~수천 개가량의 많은 코어
코어별 속도	빠름	느림
주된 연산 처리 방식	직렬 처리 방식으로 복잡한 처리가 가능함	많은 코어를 활용한 병렬 처리 방식
단점	빠른 병렬 연산이 어려움	병렬 연산 이외의 복잡한 처리가 어려움

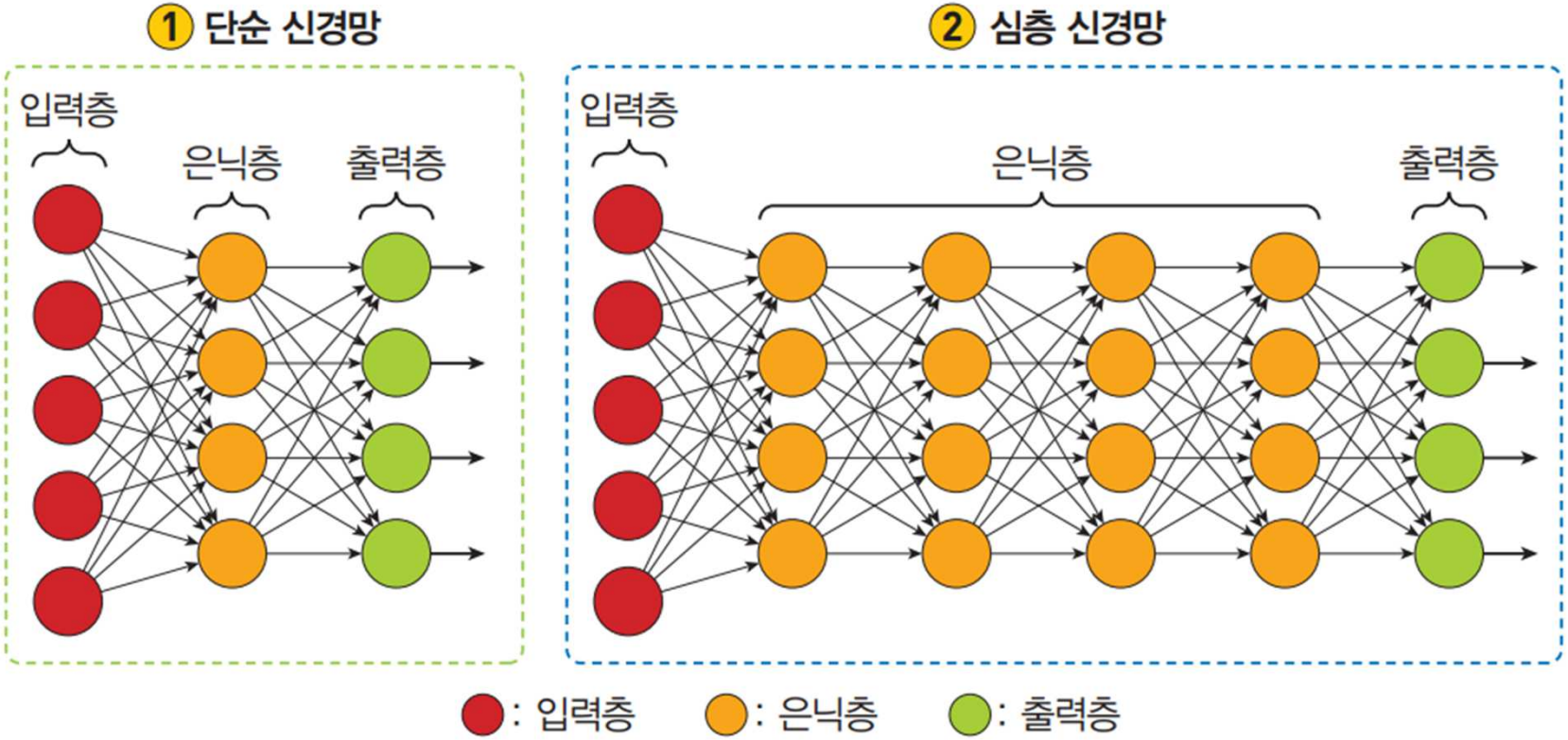
동시에 여러 가지 작업을 처리하는 병렬 처리 시스템

- 엔비디아사는 본래 비디오 게임에서 요구되는 대량의 복잡한 계산을 수행하거나 현실감 넘치는 가상현실 게임, 화려한 그래픽의 할리우드 영화 등 인간의 상상력을 그래픽으로 구현하는 데 핵심적인 역할을 해왔다.
  - 최근 빠른 딥러닝 연산 처리를 장점으로 하는 GPU의 기능을 확장하여 인공지능 하드웨어 칩을 개발하였으며 테슬라의 자율주행 기능을 담당하는 하드웨어를 개발함.



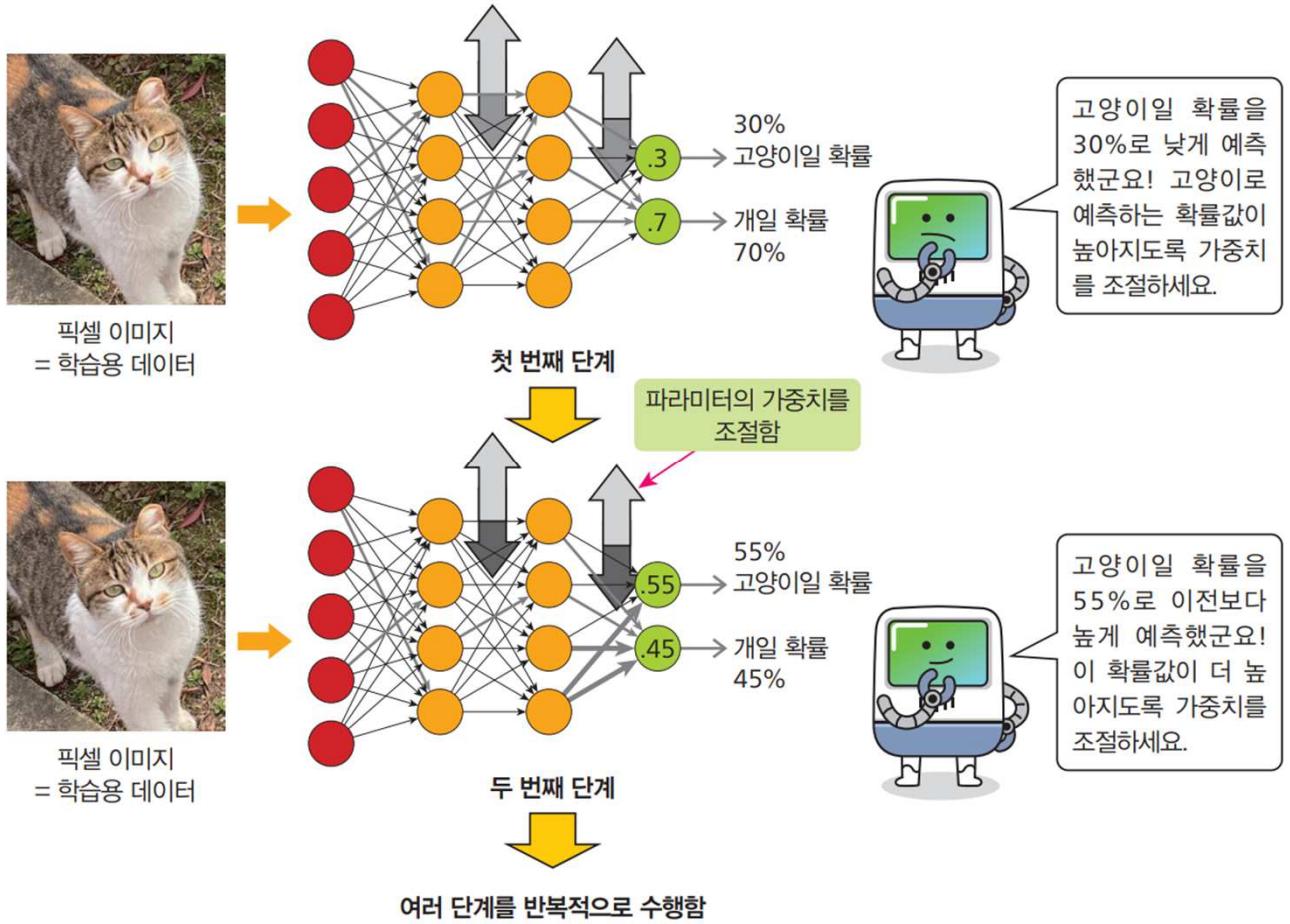
다층 퍼셉트론의 가중치들은 어떻게 학습시킬까

- 빠른 그래픽 카드, 병렬처리, 행렬연산에도 불구하고 수 없이 많은 층을 가진 심층 신경망의 학습은 어려운 일이다.
  - **딥러닝**deep learning은 이러한 심층 신경망을 학습시켜 복잡한 일을 처리할 수 있도록 함



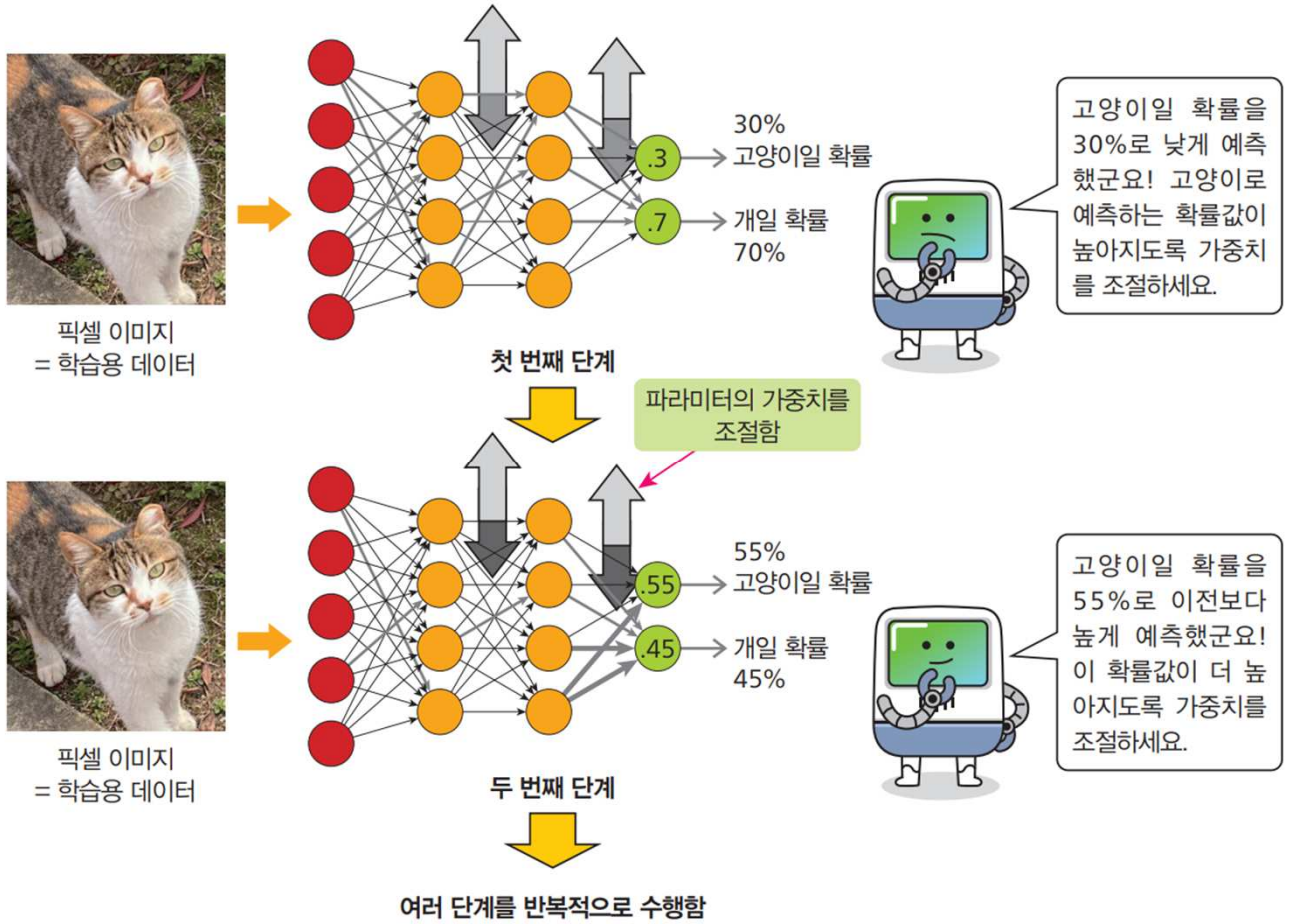
다층 퍼셉트론의 가중치들은 어떻게 학습시킬까

- 픽셀 이미지로 제공되는 동물 이미지에 대하여 고양이와 개로 구분하는 심층 신경망을 생각해 보자.
  - 이 신경망은 입력으로 픽셀 이미지가 주어지고, 이미지의 개별적인 픽셀이 가진 밝기 값들이 신경망의 입력으로 들어갈 것이다.



다층 퍼셉트론의 가중치들은 어떻게 학습시킬까

- 첫 번째 단계에서 이 픽셀 값들이 여러 층의 노드를 통과하면서 가중치들과 곱해지고 최종적으로 고양이일 확률이 30%(.3으로 표시), 개일 확률이 70%(.7로 표시)라는 결과를 출력하였다고 가정하자.
- 그렇게 되면 다음 단계에서 고양이로 예측하는 확률값이 더 높아지도록 알고리즘이 수행될 것이고, 두 번째 단계에서는 고양이일 확률이 55%, 개일 확률이 45%라는 결과를 출력하게 될 것.
- 이 과정을 반복하는 신경망 구축하기



다층 퍼셉트론의 가중치들은 어떻게 학습시킬까

- 이렇게 여러 개의 층을 가진 복잡한 퍼셉트론들을 하나하나 학습시키는 것이 쉬운 일이 아니라는 점이다.
  - 앞의 그림에서는 연결 강도의 가중치 ω를 조절하는 것이 학습의 과정이라고 나타나 있지만 어떻게 이를 조절할지는 매우 어려운 문제이다.
  - 이 어려운 문제가 해결되지 않았던 오랜 기간 동안 인공지능 연구는 침체기를 맞이하였는데, 이 문제의 획기적인 돌파구가 된 것이 바로 **오차 역전파 알고리즘**이다.

요약

01 인간과 동물이 가진 **뉴런이라는 신경세포는 다른 신경세포와 연결되어 있고**, 나트륨, 칼륨 등의 물질을 이용하여 화학적 신호를 주고받으며 정보를 전달한다.

02 하나의 **신경세포** 구조를 **흉내 내는 컴퓨터 프로그램**을 컴퓨터 과학자들은 **인공 신경세포** 혹은 **퍼셉트론**이라고 한다.

03 퍼셉트론 구조에서 외부 자극에 곱해지는 값은 **파라미터** 혹은 **가중치**로 부르는데 이것은 신경망 학습의 핵심이 된다.

04 활성화 함수에 입력된 신호 합산 값을 다음 퍼셉트론에 전달할지 말지를 결정하기 위해 **임계값** 혹은 **문턱값**이라고 하는 상수를 사용하며, 이 값은 신경세포에 전달된 신호의 누적 합이 특정한 자극치를 넘어야만 이웃 신경세포로 전달하는 역할을 담당한다.

05 비선형 함수를 만드는 방법 중의 하나로 입력층과 출력 사이에 신호를 중계하는 노드를 두는데 이러한 노드를 **은닉 노드**라고 하며 이들의 계층을 **은닉층**이라고 부른다.

06 **헵의 학습 법칙**이란 두 개의 이웃 세포가 자극에 의해 활성화되면 이 연결은 **강화**된다는 법칙이다.

07 퍼셉트론의 학습은 **헵의 학습 법칙**에 근거를 두고 있으나, 두 신경세포의 활성화가 아니라, **신경세포가 달성해야 할 목표치를 제시하고 가중치가 목표치를 만족시키도록 만들어 간다는 점에서 차이**가 있다.

08 **다층 퍼셉트론의 출력값**을 구하는 방향과 반대 방향으로 전파되어 **가중치를 계산하는 과정**을 **역전파** 과정이라고 한다.

09 **다층 신경망은 단층 신경망**과는 달리 입력 신호를 바로 출력층으로 내보내지 않고 **입력 신호를 받아 숨겨진 층을 거친 뒤에 출력층으로 보내는 신경망**이다. 이것을 **더욱 복잡하게 만들면 많은 층을 거쳐 가는 깊은 신경망**이 될 것이고, 이는 **딥러닝** 개념의 시작이라고 할 수 있다.